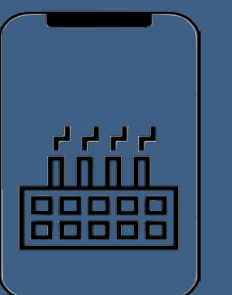


Sorting Algorithms



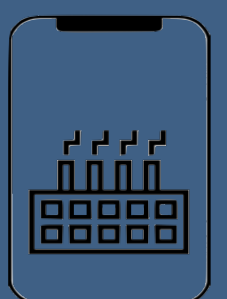
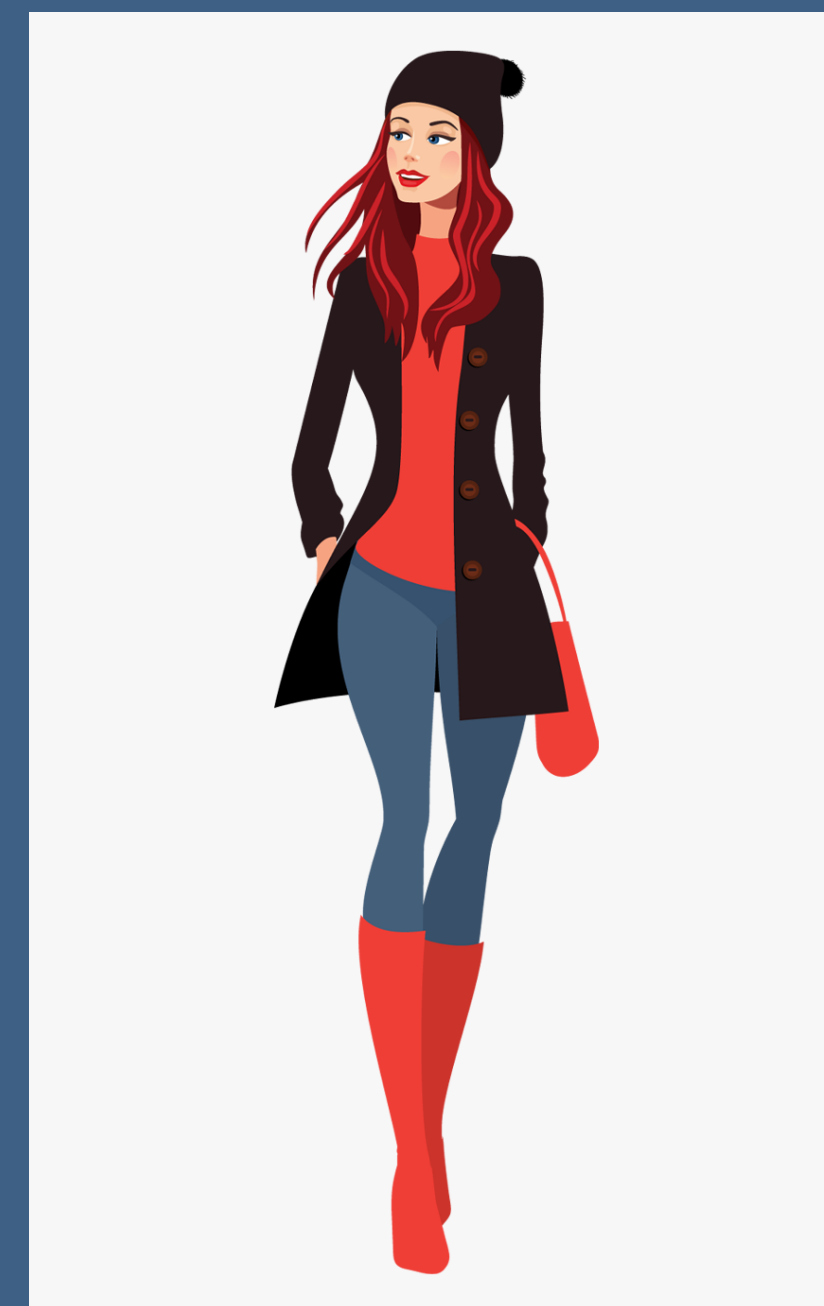
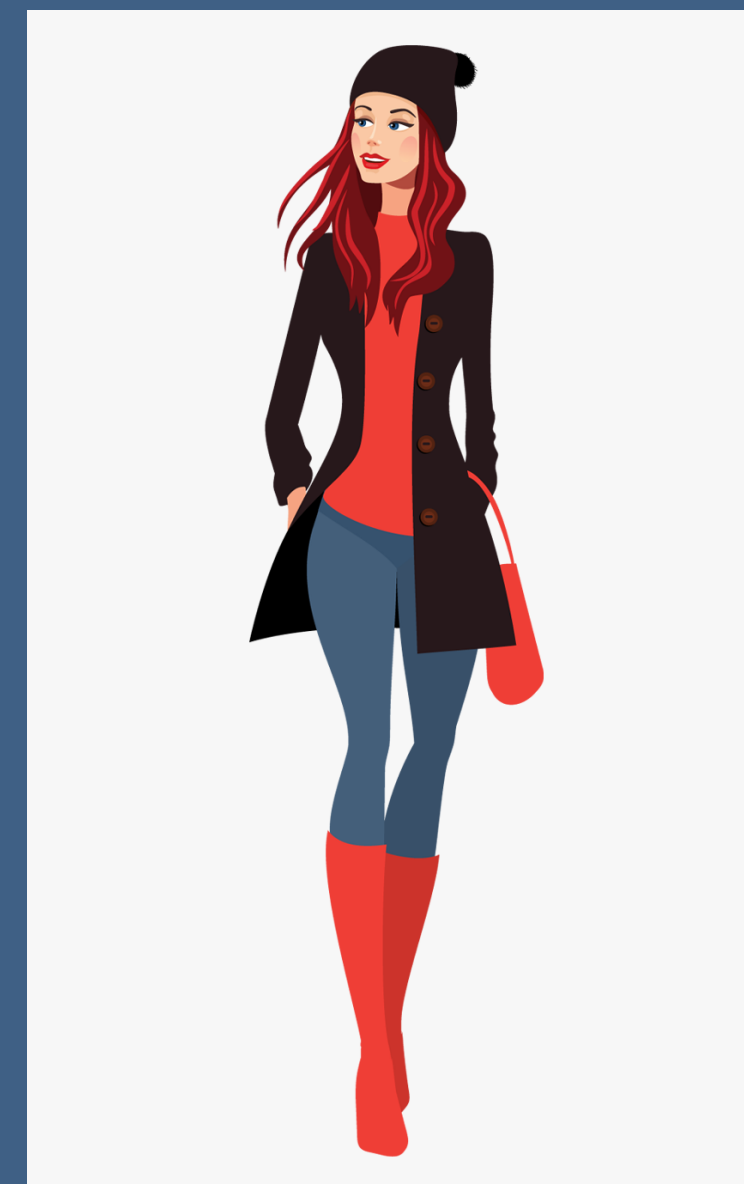
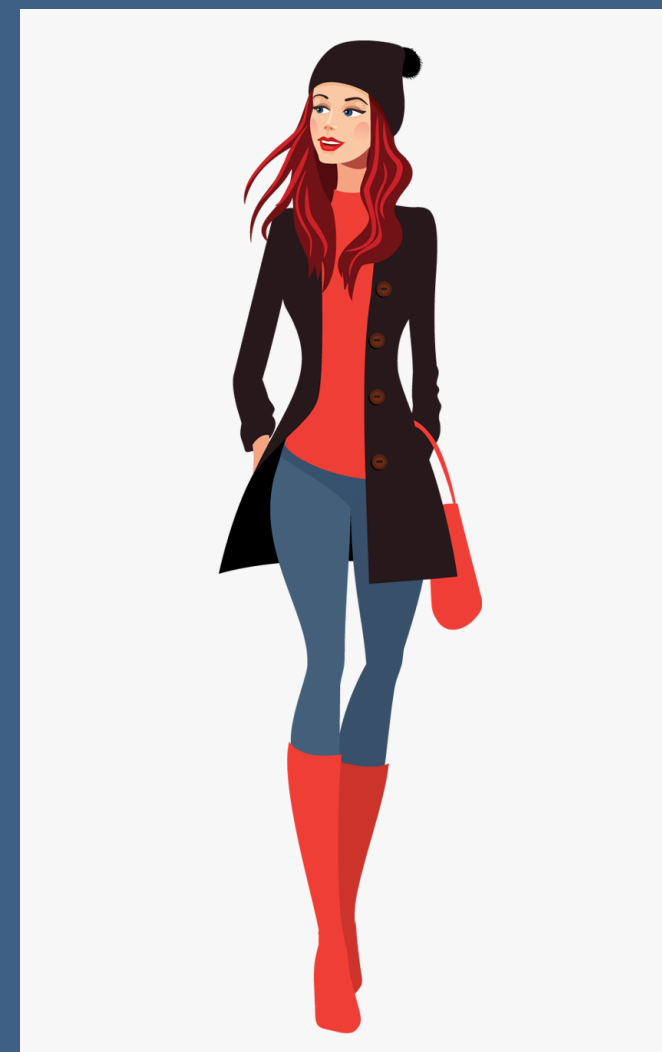
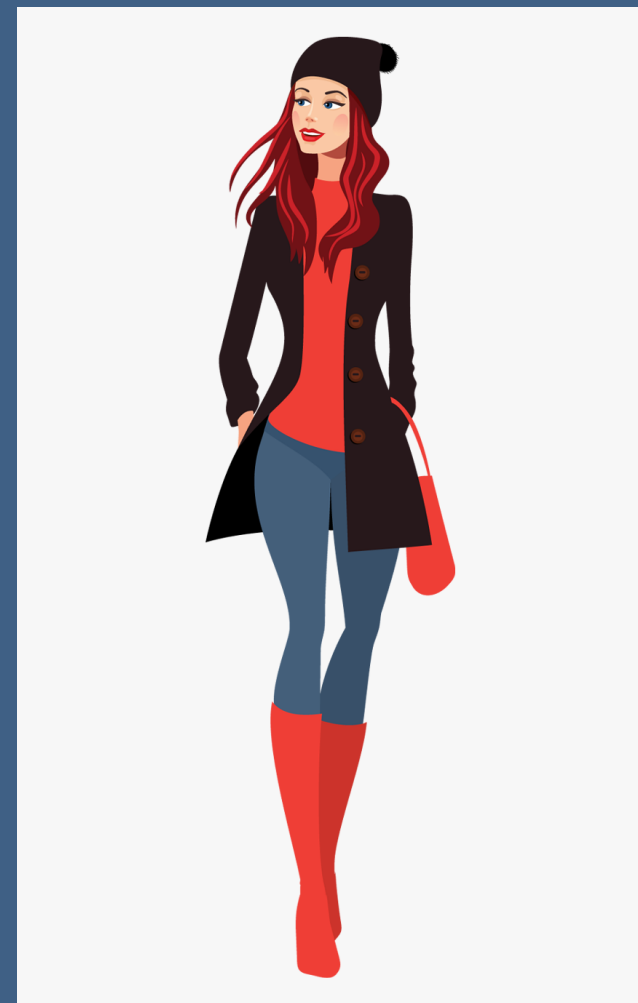
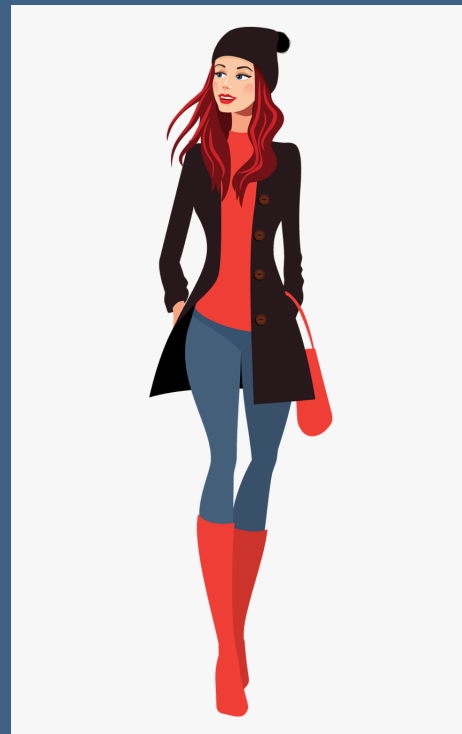
What is Sorting?

By definition sorting refers to arranging data in a particular format : either ascending or descending.



What is Sorting?

By definition sorting refers to arranging data in a particular format : either ascending or descending.

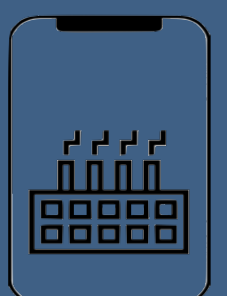
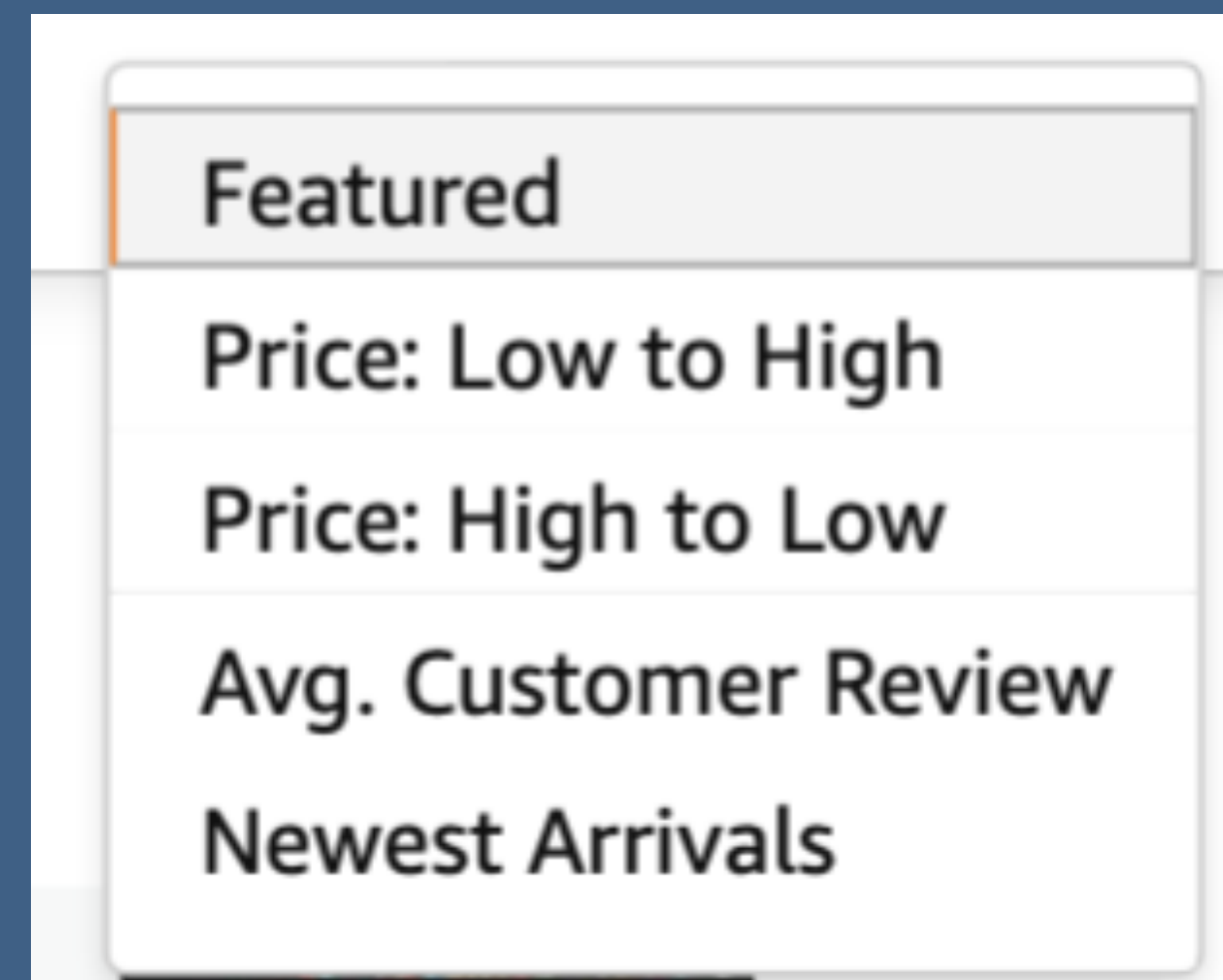
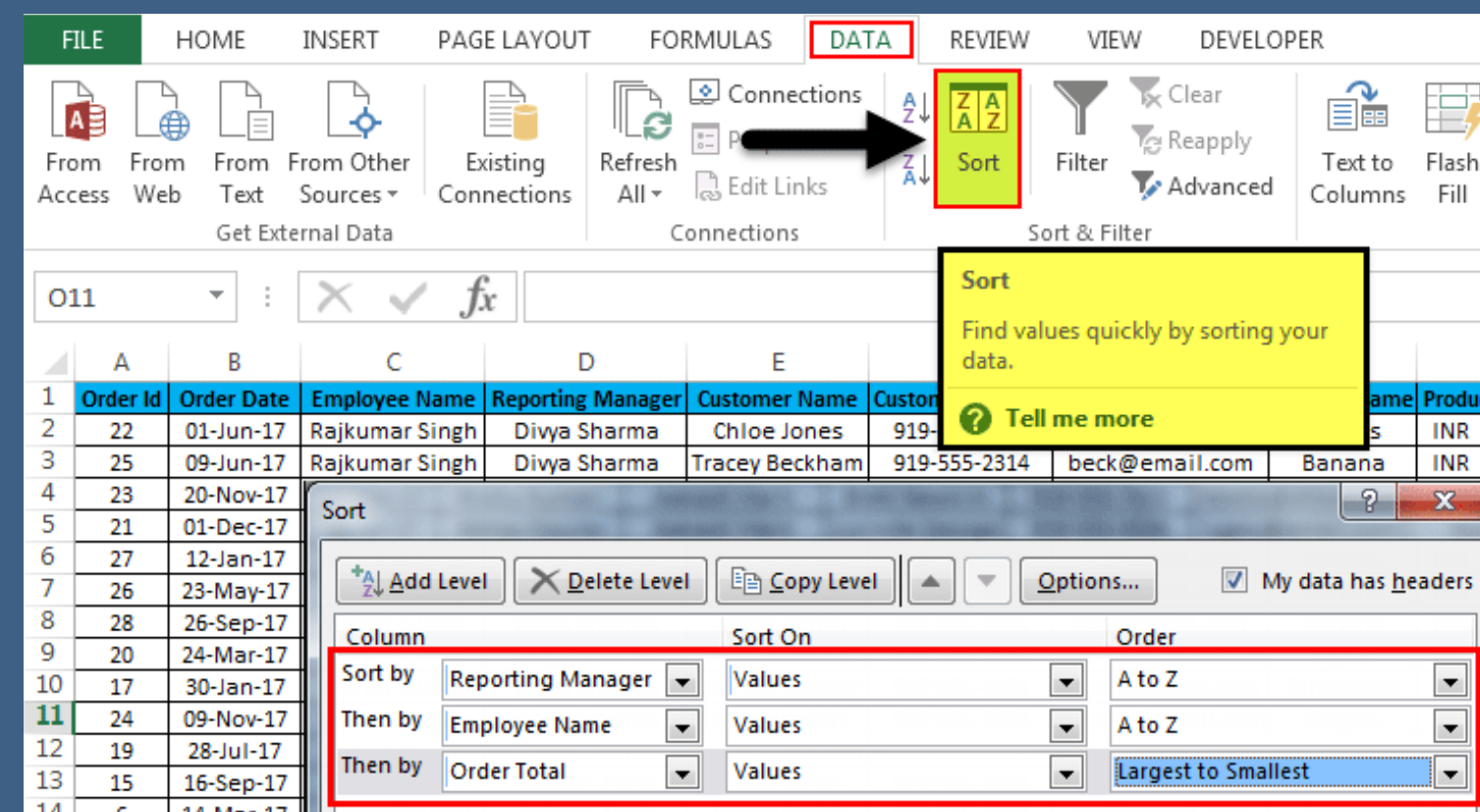


What is Sorting?

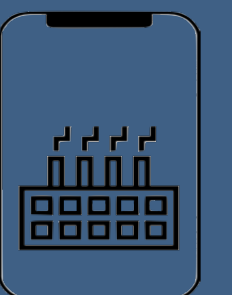
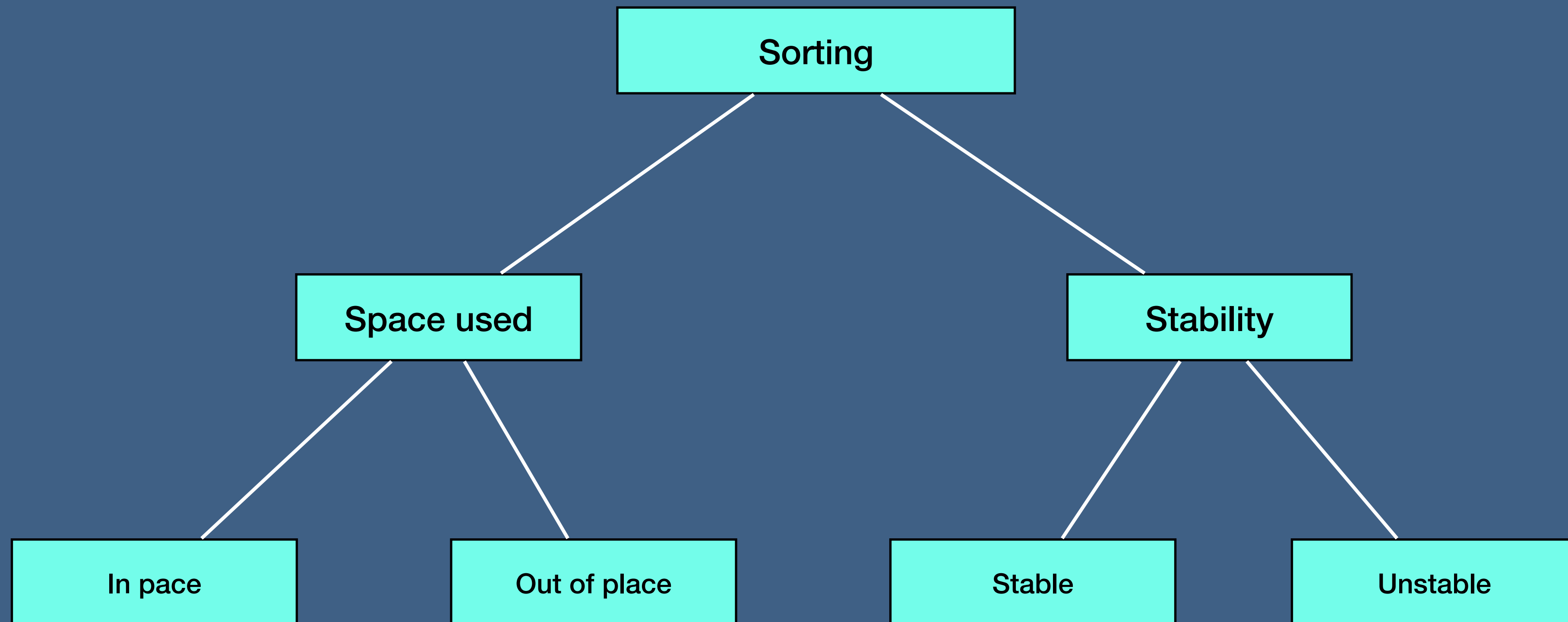
Practical Use of Sorting

Microsoft Excel : Built in functionality to sort data

Online Stores: Online shopping websites generally have option for sorting by price, review, ratings..



Types of Sorting



Types of Sorting

Space used

In place sorting : Sorting algorithms which does not require any extra space for sorting

Example : Bubble Sort

70	10	80	30	20	40	60	50	90
----	----	----	----	----	----	----	----	----

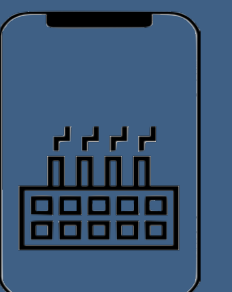
10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----

Out place sorting : Sorting algorithms which requires an extra space for sorting

Example : Merge Sort

70	10	80	30	20	40	60	50	90
----	----	----	----	----	----	----	----	----

10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----



Types of Sorting

Stability

Stable sorting : If a sorting algorithm after sorting the contents does not change the sequence of similar content in which they appear, then this sorting is called stable sorting.

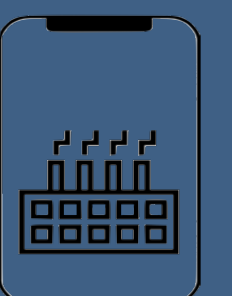
Example : Insertion Sort

70	10	80	40	20	40	60	50	90
10	20	40	40	50	60	70	80	90

UnStable sorting : If a sorting algorithm after sorting the content changes the sequence of similar content in which they appear, then it is called unstable sort.

Example : Quick Sort

70	10	80	40	20	40	60	50	90
10	20	40	40	50	60	70	80	90



Types of Sorting

Stability

UnStable sorting example

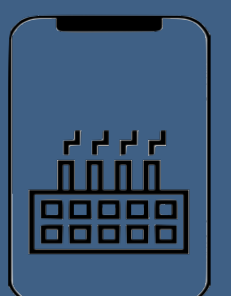
Unsorted data	
Name	Age
Renad	7
Nick	6
Richard	6
Parker	7
Sofia	7

Sorted by name	
Name	Age
Nick	6
Parker	7
Renad	7
Richard	6
Sofia	7



Sorted by age (stable)	
Name	Age
Nick	6
Richard	6
Parker	7
Renad	7
Sofia	7

Sorted by age (unstable)	
Name	Age
Nick	6
Richard	6
Renad	7
Parker	7
Sofia	7



Sorting Terminology

Increasing Order

- If successive element is greater than the previous one
- Example : 1, 3, 5, 7, 9, 11

Decreasing Order

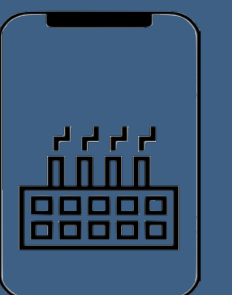
- If successive element is less than the previous one
- Example : 11, 9, 7, 5, 3, 1

Non Increasing Order

- If successive element is less than or equal to its previous element in the sequence.
- Example : 11, 9, 7, 5, 5, 3, 1

Non Decreasing Order

- If successive element is greater than or equal to its previous element in the sequence
- Example : 1, 3, 5, 7, 7, 9, 11

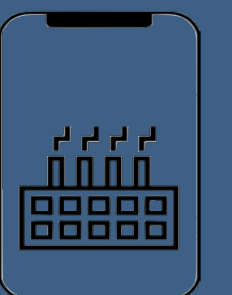


Sorting Algorithms

Bubble sort
Selection sort
Insertion sort
Bucket sort
Merge sort
Quick sort
Heap sort

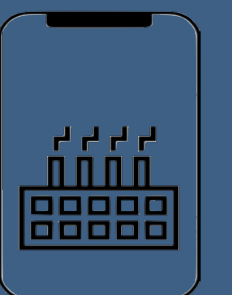
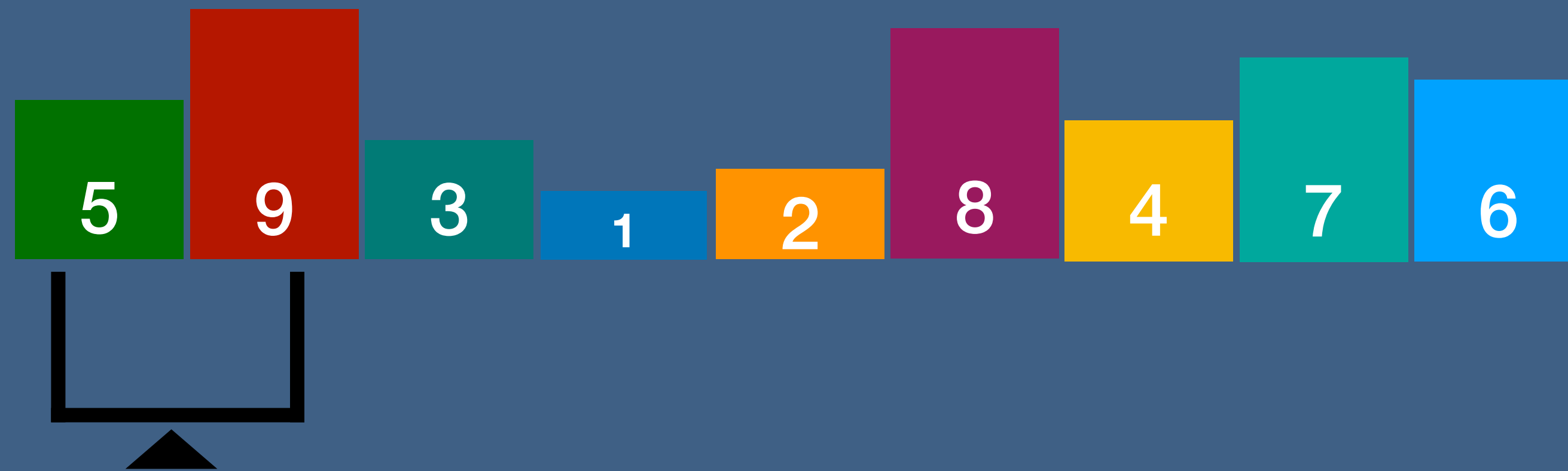
Which one to select?

- Stability
- Space efficient
- Time efficient



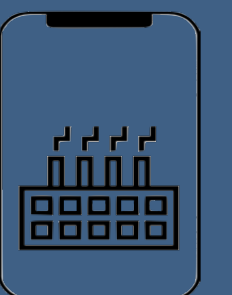
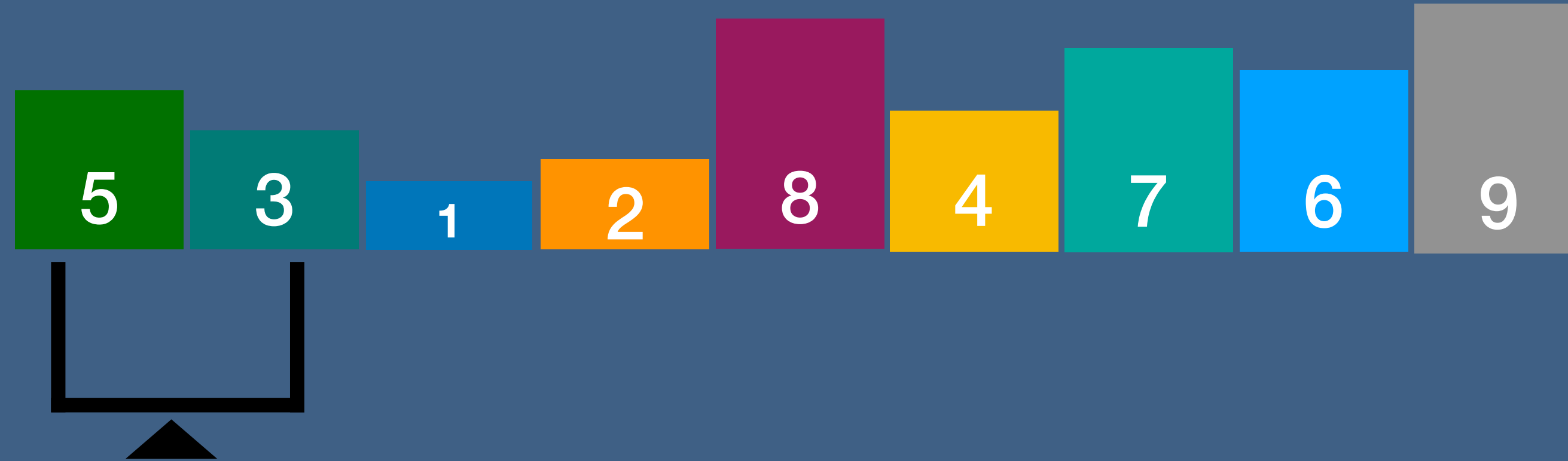
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



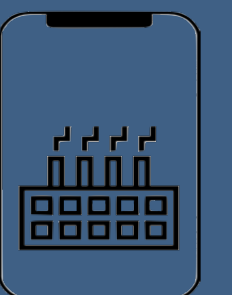
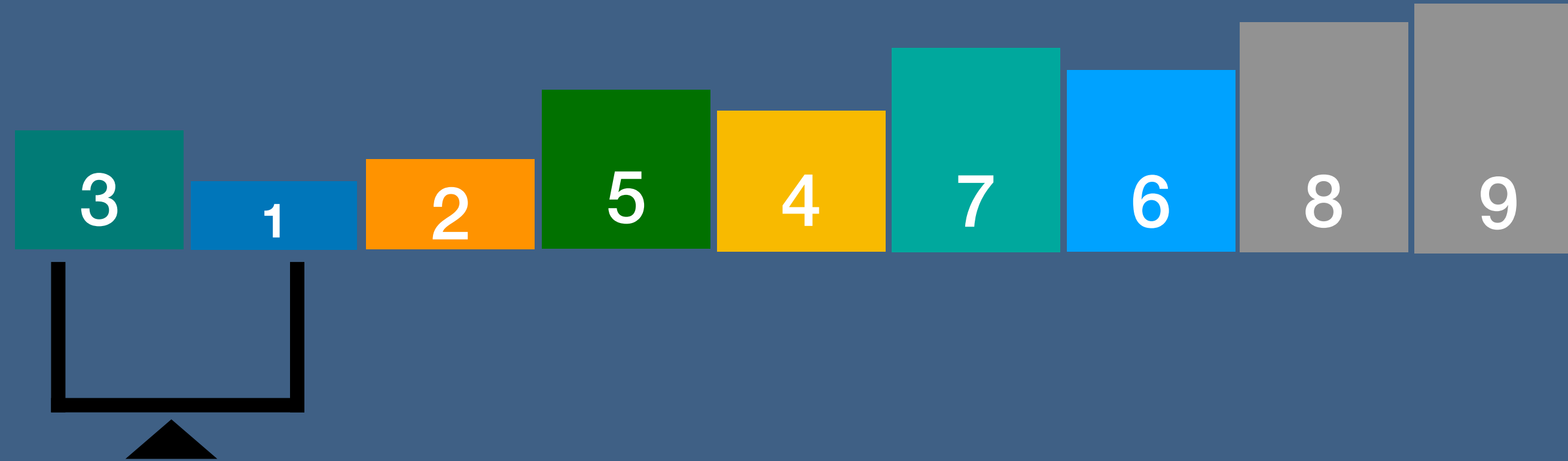
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



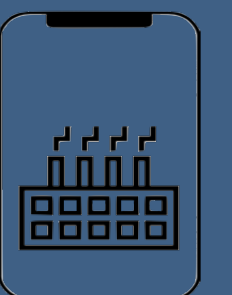
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



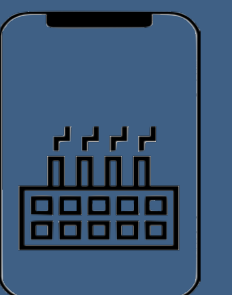
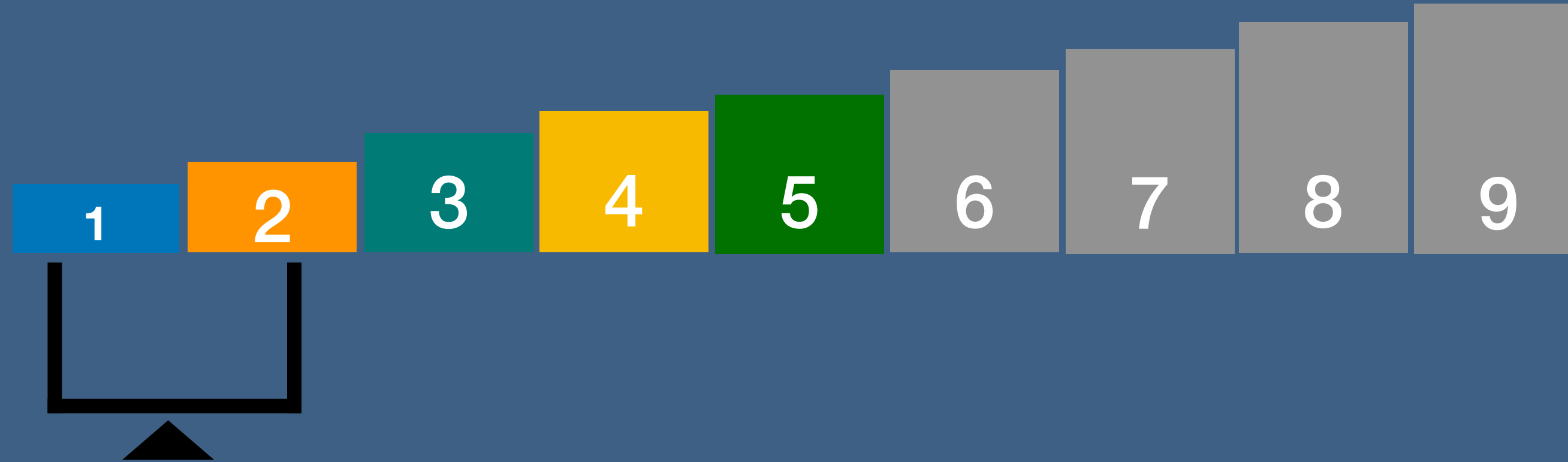
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



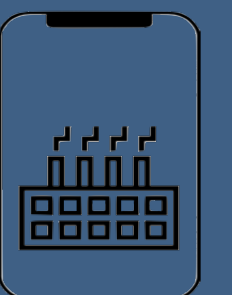
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



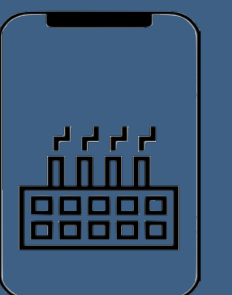
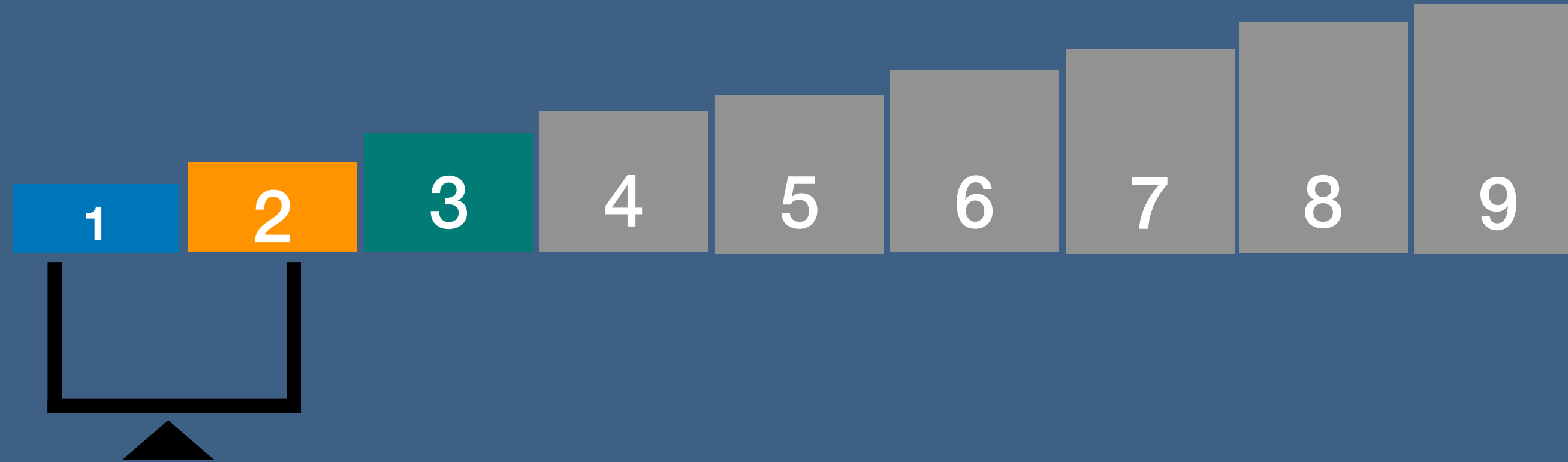
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



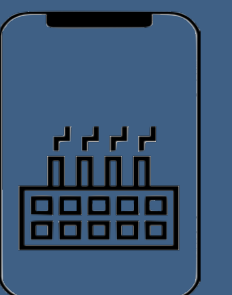
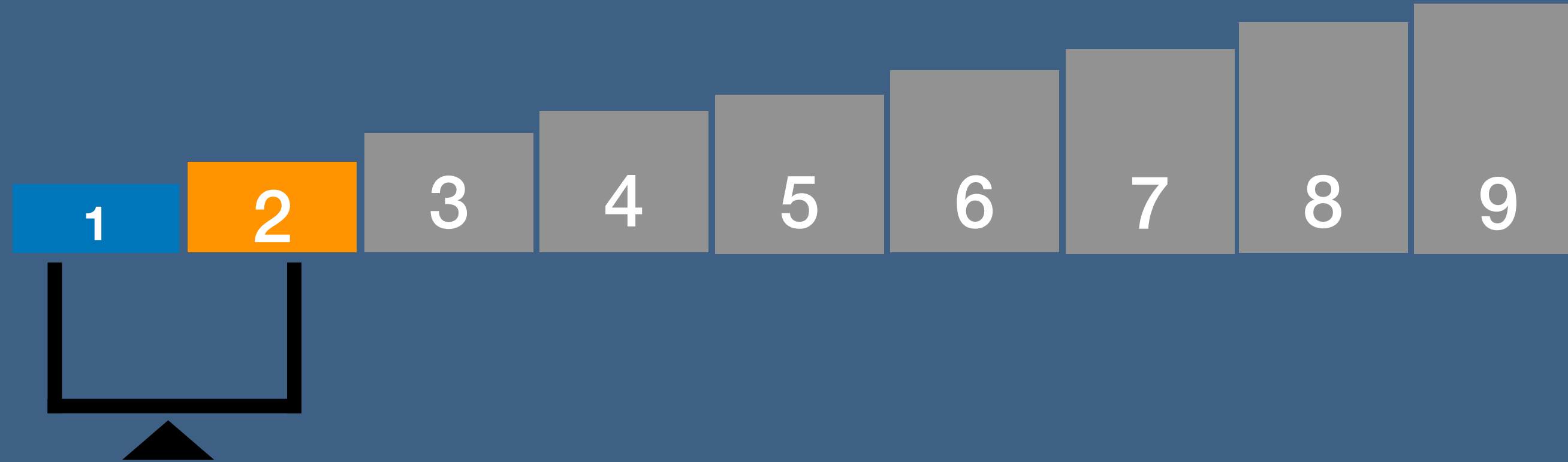
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



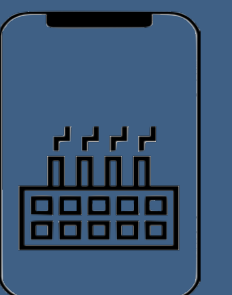
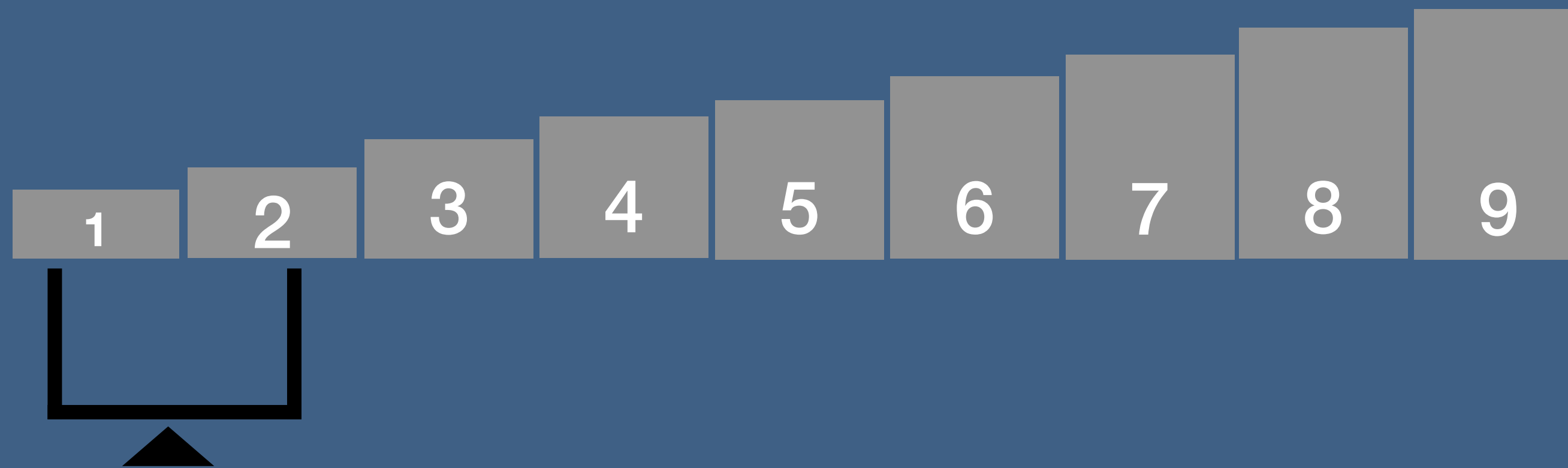
Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



Bubble Sort

- Bubble sort is also referred as Sinking sort
- We repeatedly compare each pair of adjacent items and swap them if they are in the wrong order



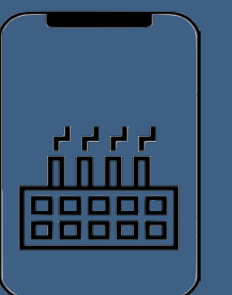
Bubble Sort

When to use Bubble Sort?

- When the input is almost sorted
- Space is a concern
- Easy to implement

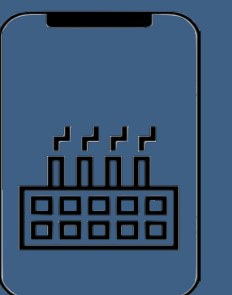
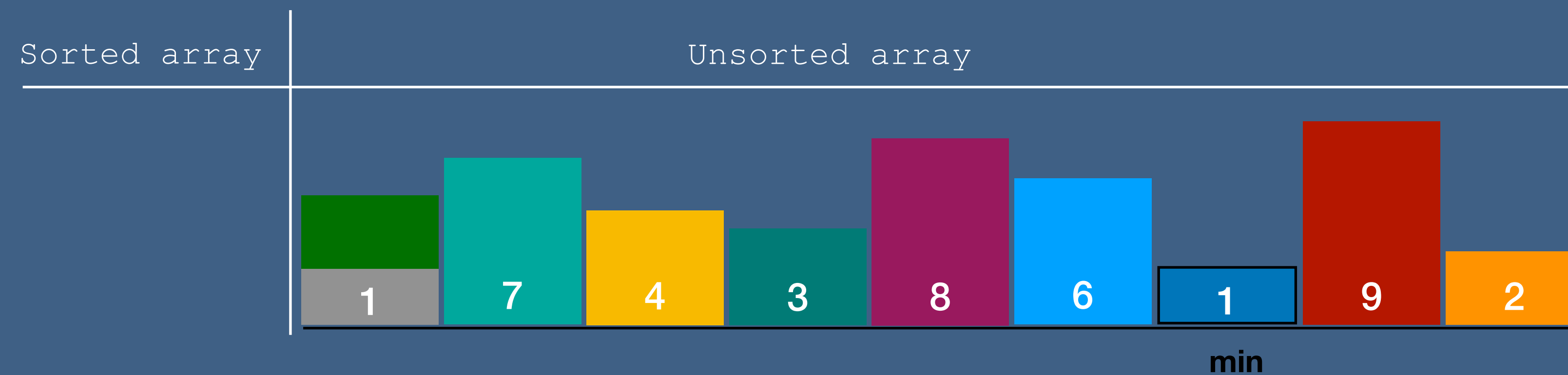
When to avoid Bubble Sort?

- Average time complexity is poor



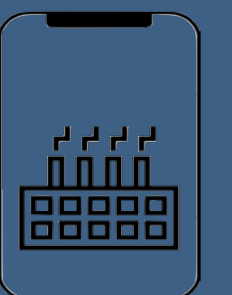
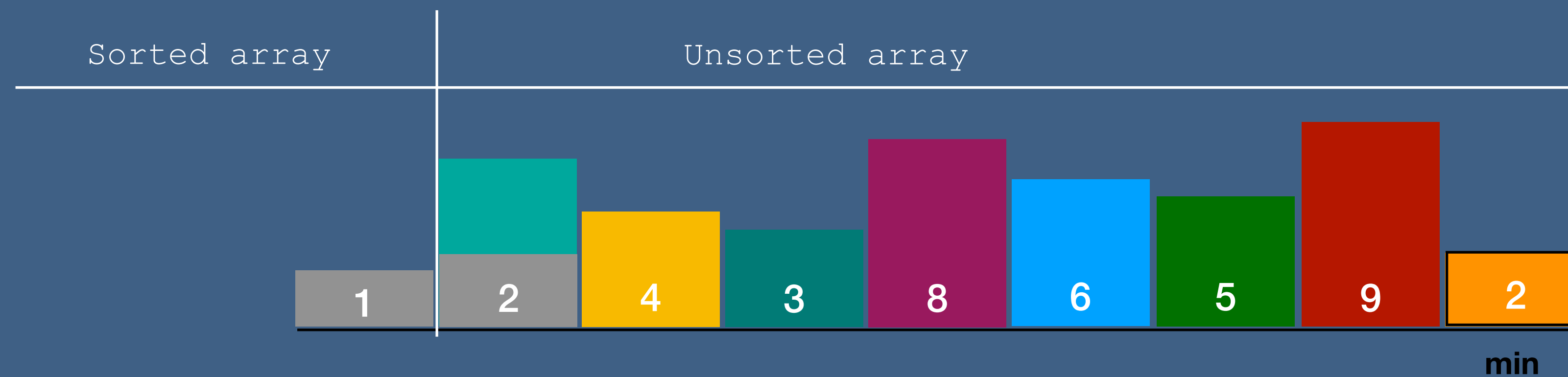
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



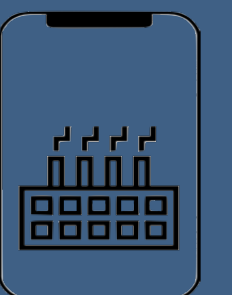
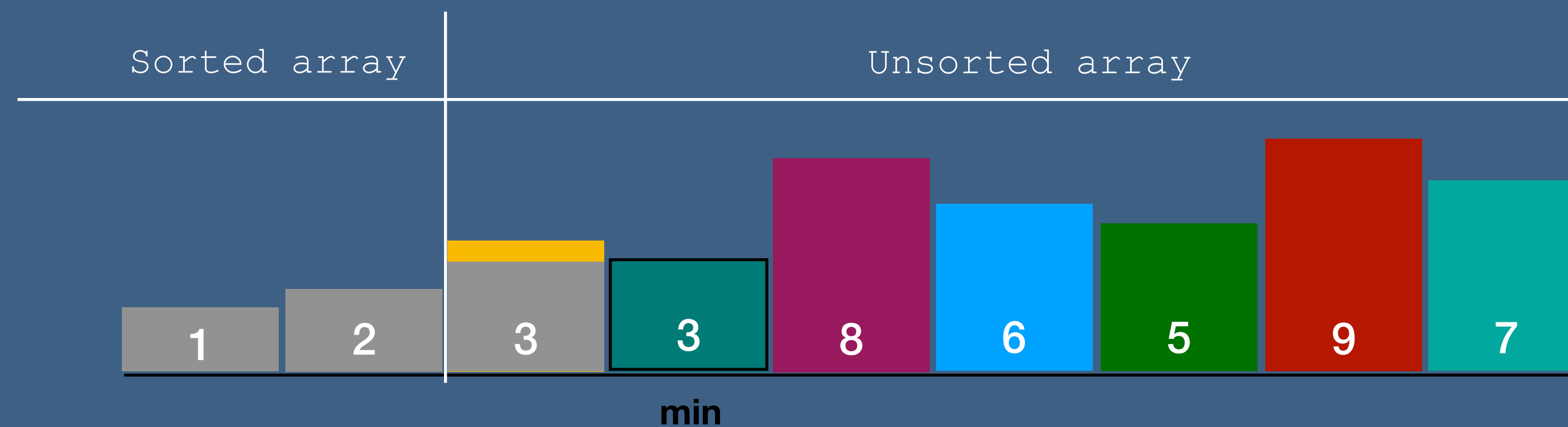
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



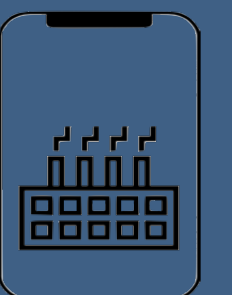
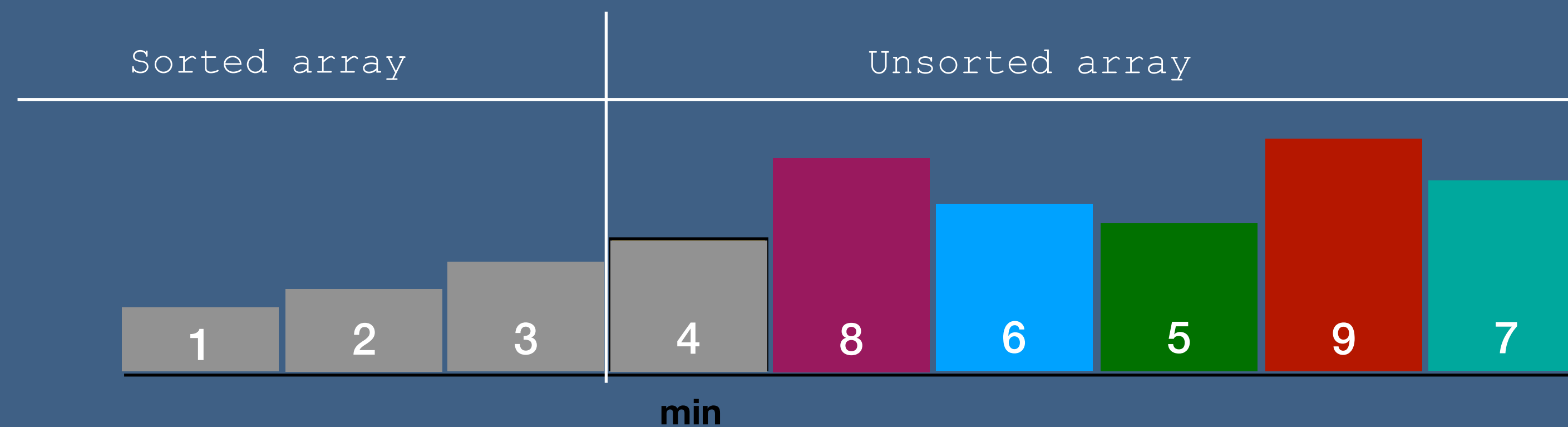
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



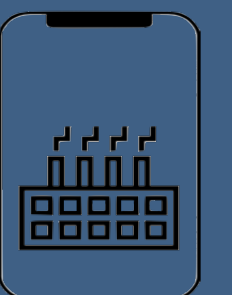
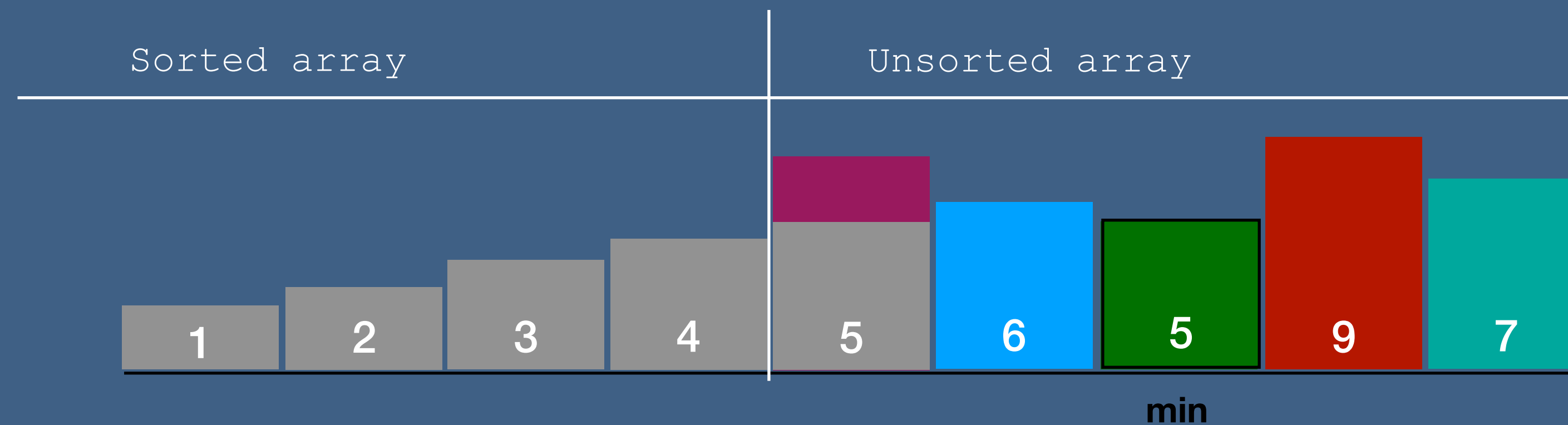
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



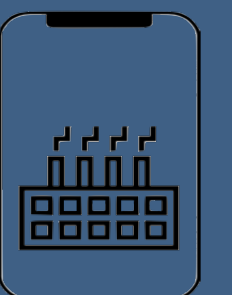
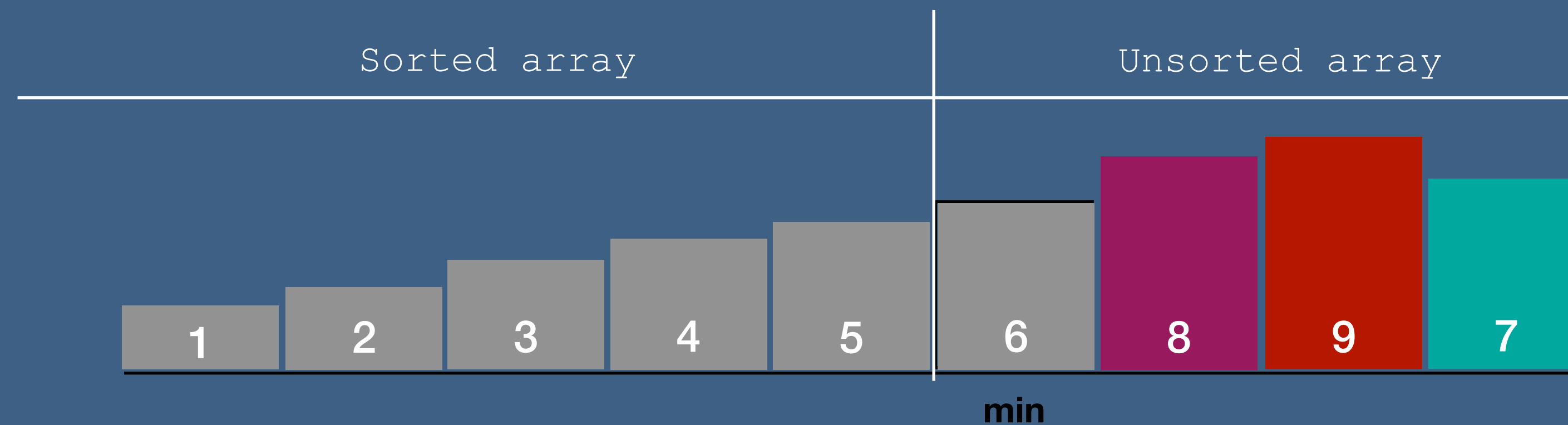
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



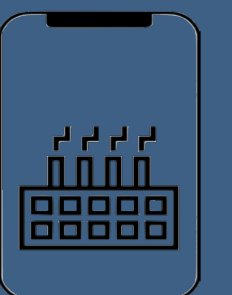
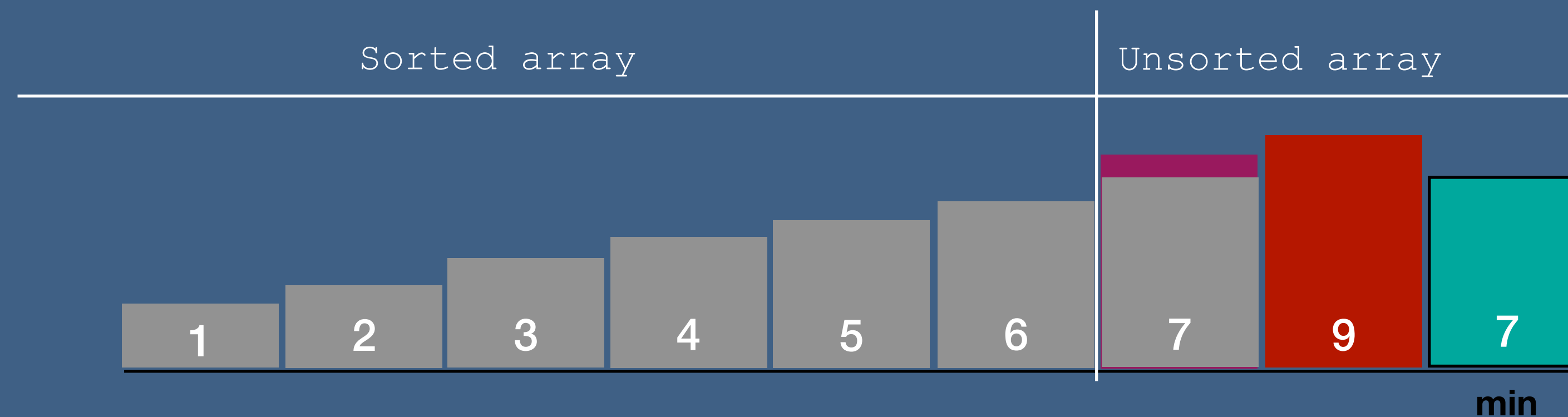
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



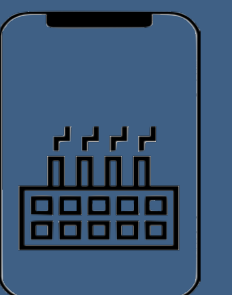
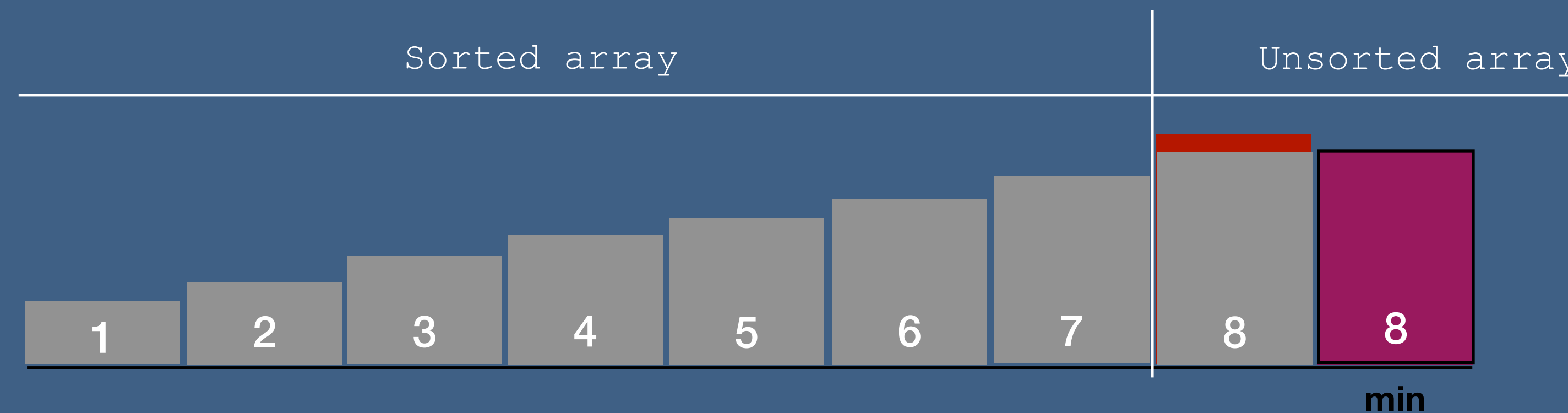
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



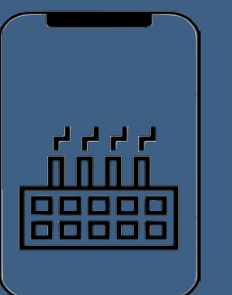
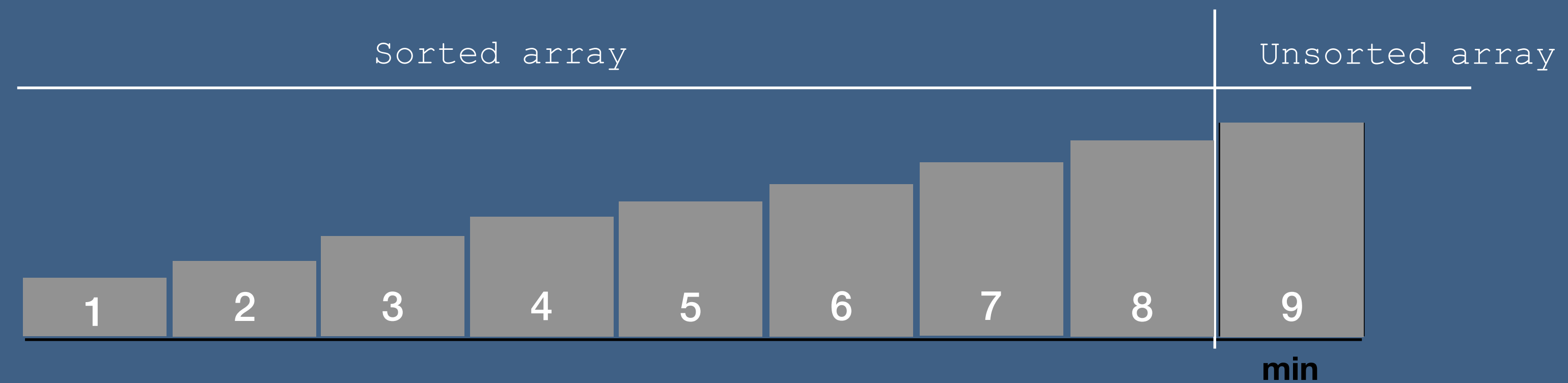
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



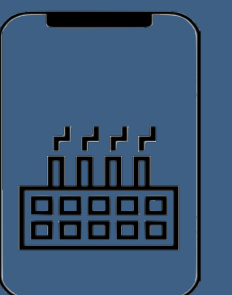
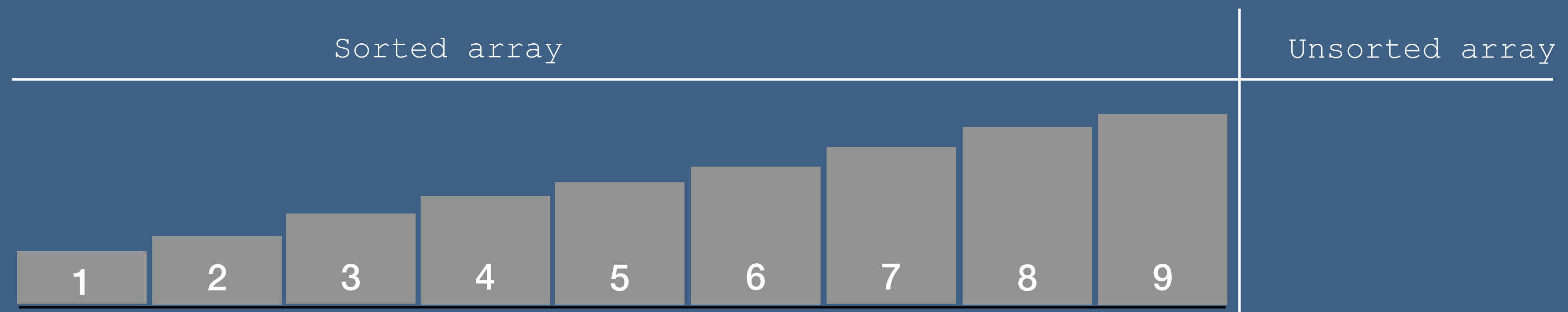
Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



Selection Sort

- In case of selection sort we repeatedly find the minimum element and move it to the sorted part of array to make unsorted part sorted.



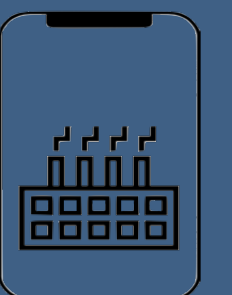
Selection Sort

When to use Selection Sort?

- When we have insufficient memory
- Easy to implement

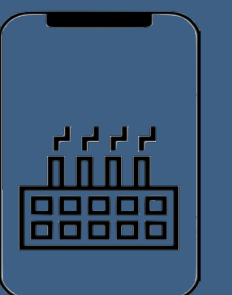
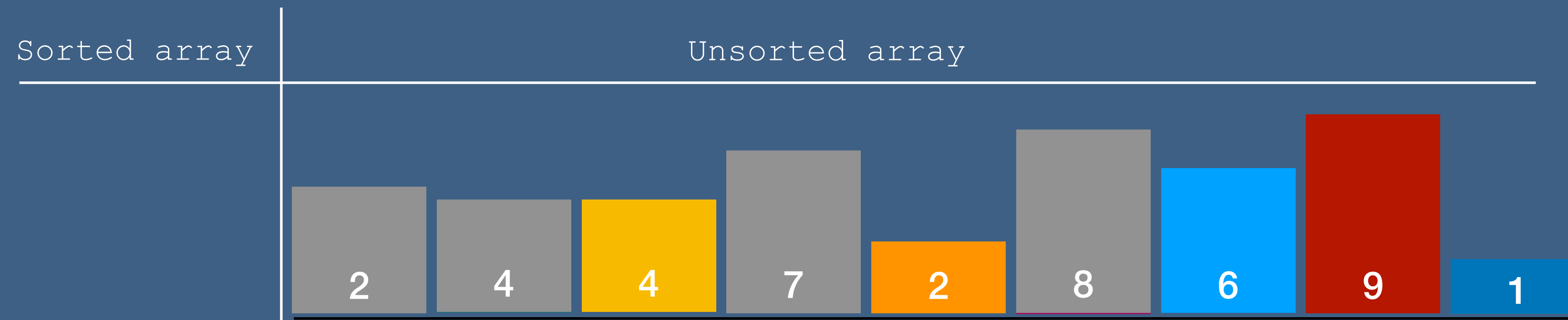
When to avoid Selection Sort?

- When time is a concern



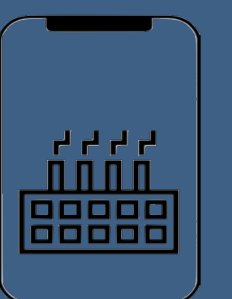
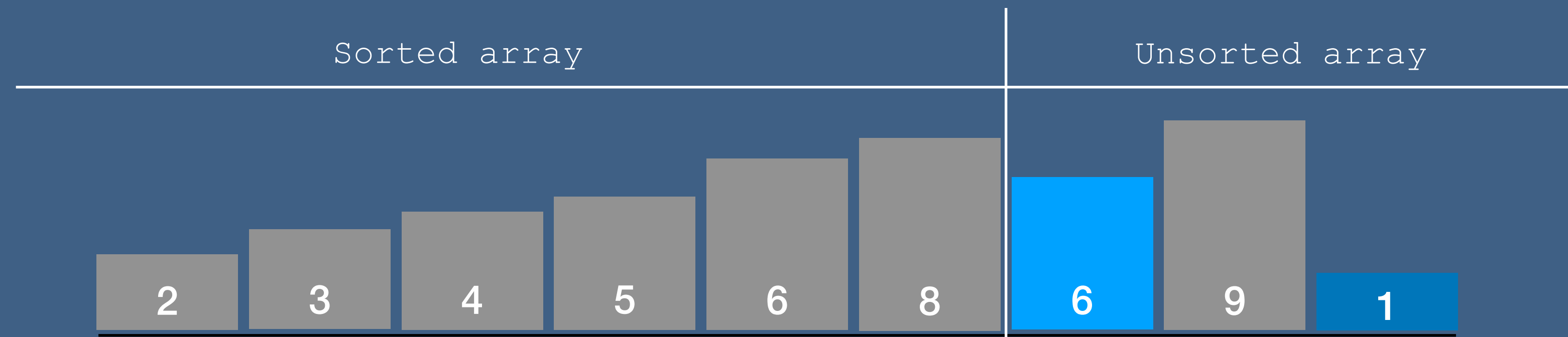
Insertion Sort

- Divide the given array into two part
- Take first element from unsorted array and find its correct position in sorted array
- Repeat until unsorted array is empty



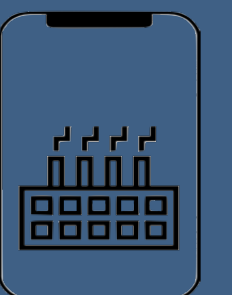
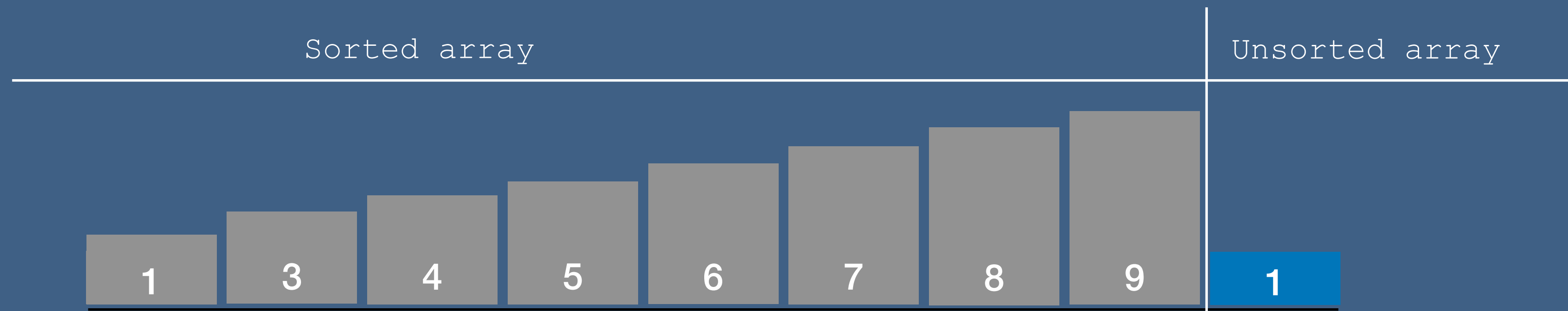
Insertion Sort

- Divide the given array into two part
- Take first element from unsorted array and find its correct position in sorted array
- Repeat until unsorted array is empty



Insertion Sort

- Divide the given array into two part
- Take first element from unsorted array and find its correct position in sorted array
- Repeat until unsorted array is empty



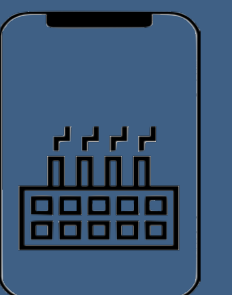
Insertion Sort

When to use Insertion Sort?

- When we have insufficient memory
- Easy to implement
- When we have continuous inflow of numbers and we want to keep them sorted

When to avoid Insertion Sort?

- When time is a concern

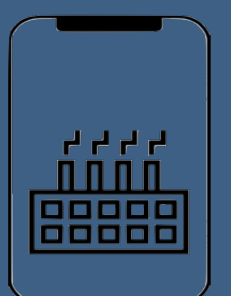
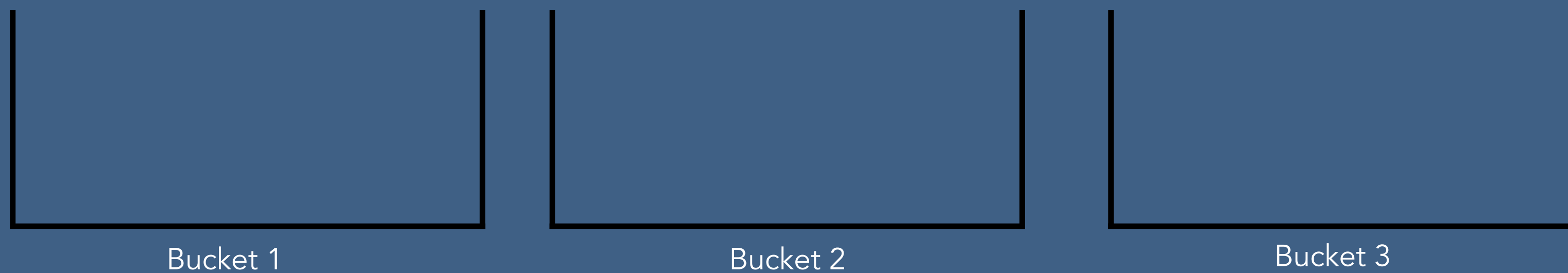


Bucket Sort

- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting



- Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$
 $\text{round}(\text{sqrt}(9)) = 3$
- Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$
 $\text{ceil}(5 * 3 / 9) = \text{ceil}(1.6) = 2$



Bucket Sort

- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting

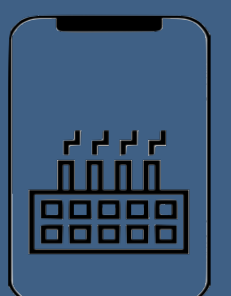
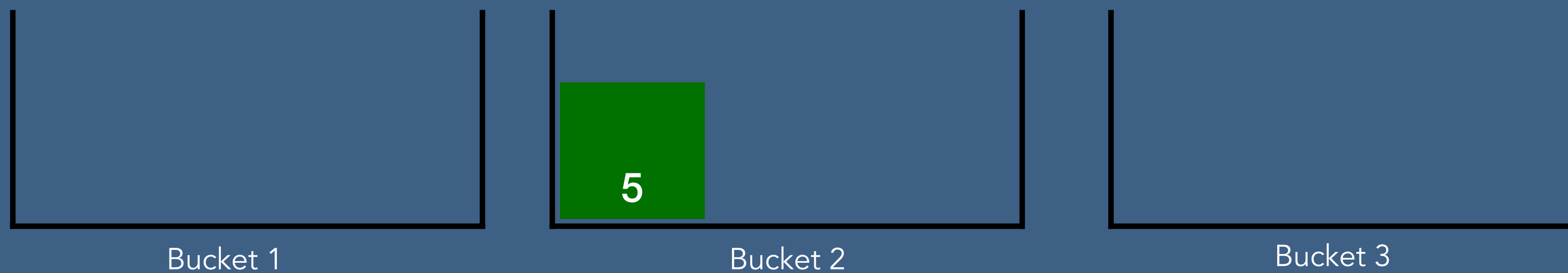


Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

$\text{round}(\text{sqrt}(9)) = 3$

Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

$\text{ceil}(4 * 3 / 9) = \text{ceil}(1.33) = 2$



Bucket Sort

- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting

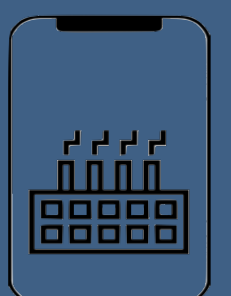
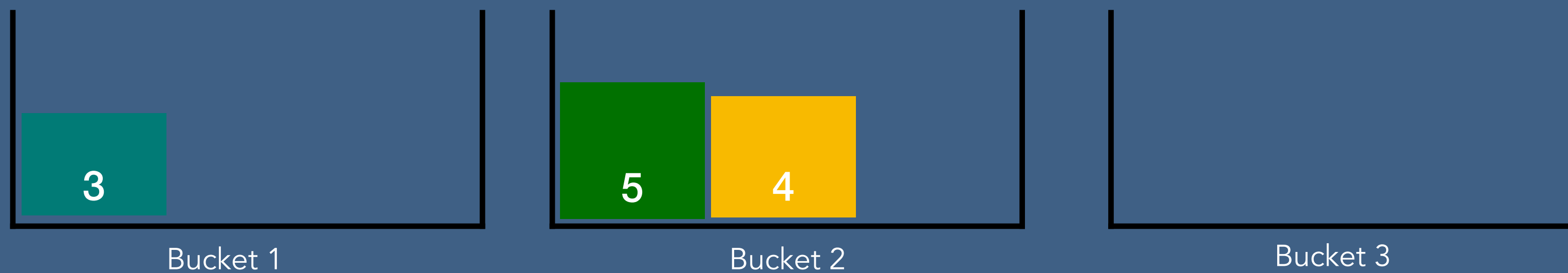


Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

$\text{round}(\text{sqrt}(9)) = 3$

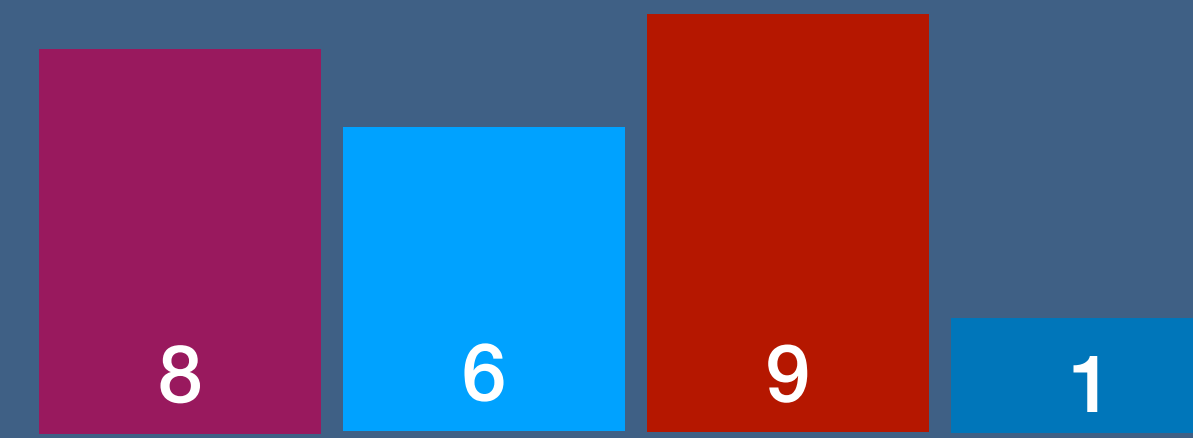
Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

$\text{ceil}(2 * 3 / 9) = \text{ceil}(0.6) = 1$



Bucket Sort

- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting

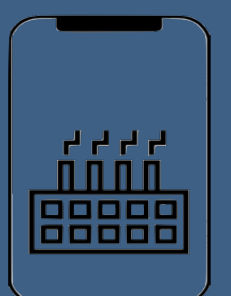
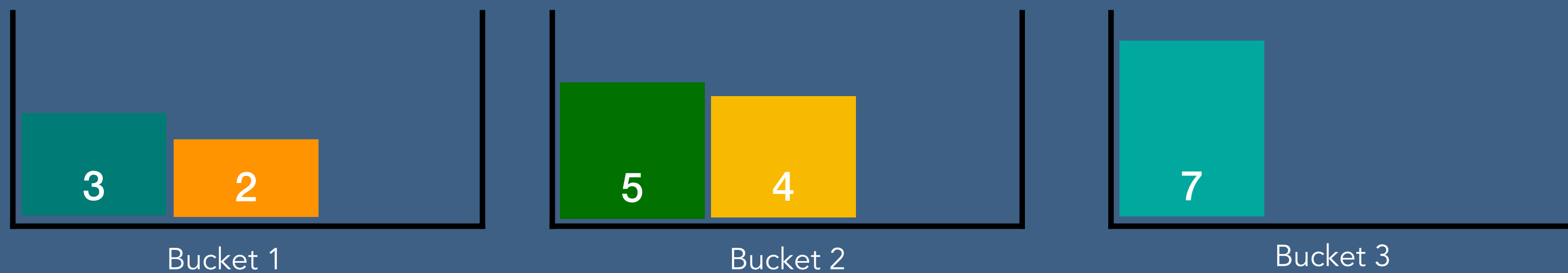


Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

$\text{round}(\text{sqrt}(9)) = 3$

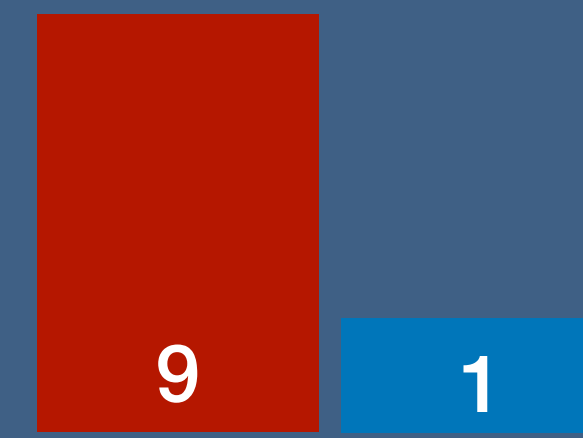
Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

$\text{ceil}(8 * 3 / 9) = \text{ceil}(2.67) = 3$



Bucket Sort

- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting



Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

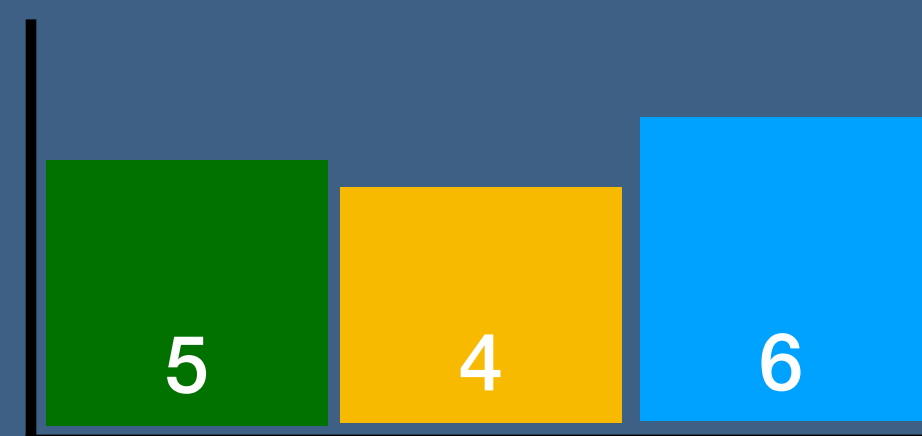
$\text{round}(\text{sqrt}(9)) = 3$

Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

$\text{ceil}(9 * 3 / 9) = \text{ceil}(3) = 3$
Sort all buckets (using any sorting algorithm)



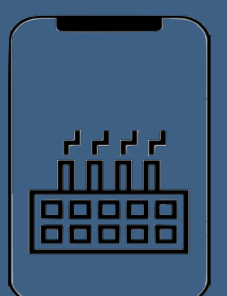
Bucket 1



Bucket 2



Bucket 3



Bucket Sort

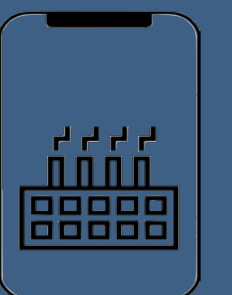
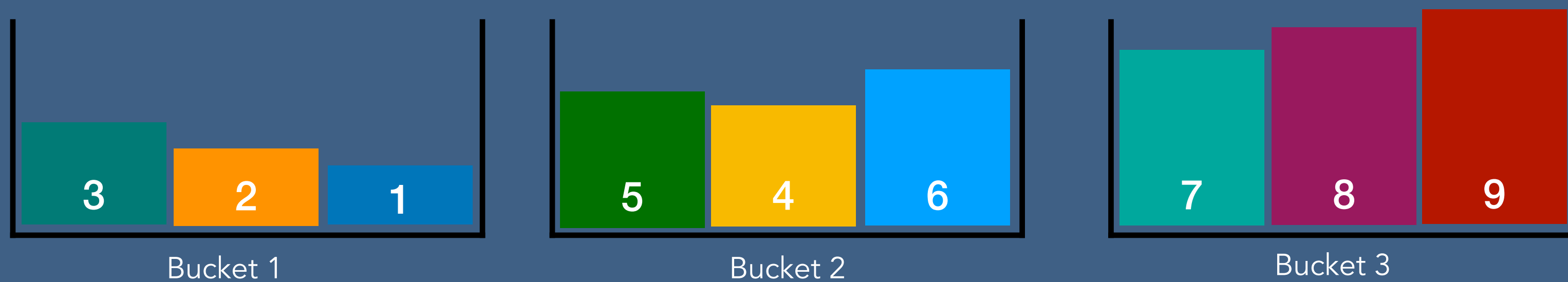
- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting

Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

$\text{round}(\text{sqrt}(9)) = 3$

Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

Sort all buckets (using any sorting algorithm)



Bucket Sort

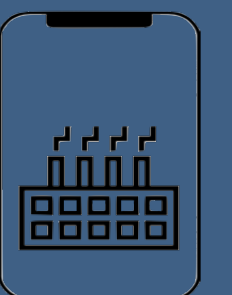
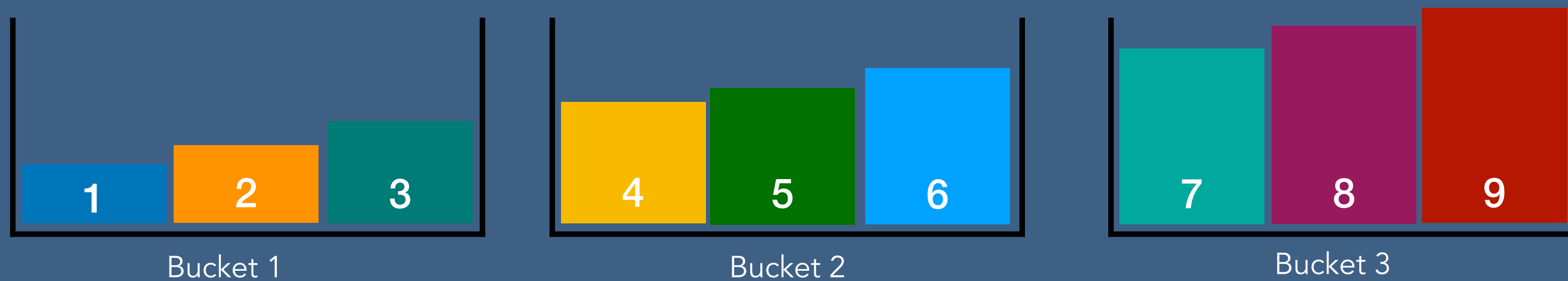
- Create buckets and distribute elements of array into buckets
- Sort buckets individually
- Merge buckets after sorting

Number of buckets = $\text{round}(\text{Sqrt}(\text{number of elements}))$

$\text{round}(\text{sqrt}(9)) = 3$

Appropriate bucket = $\text{ceil}(\text{Value} * \text{number of buckets} / \text{maxValue})$

Sort all buckets (using any sorting algorithm)



Bucket Sort

When to use Bucket Sort?

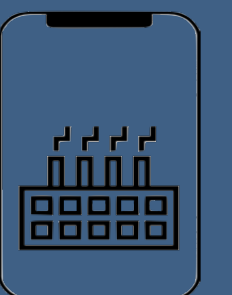
- When input uniformly distributed over range

1,2,4,5,3,8,7,9

1,2,4,91,93,95

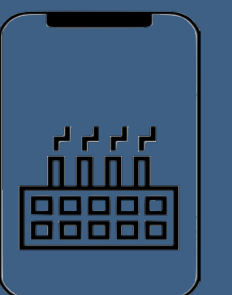
When to avoid Bucket Sort?

- When space is a concern

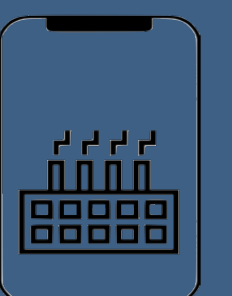
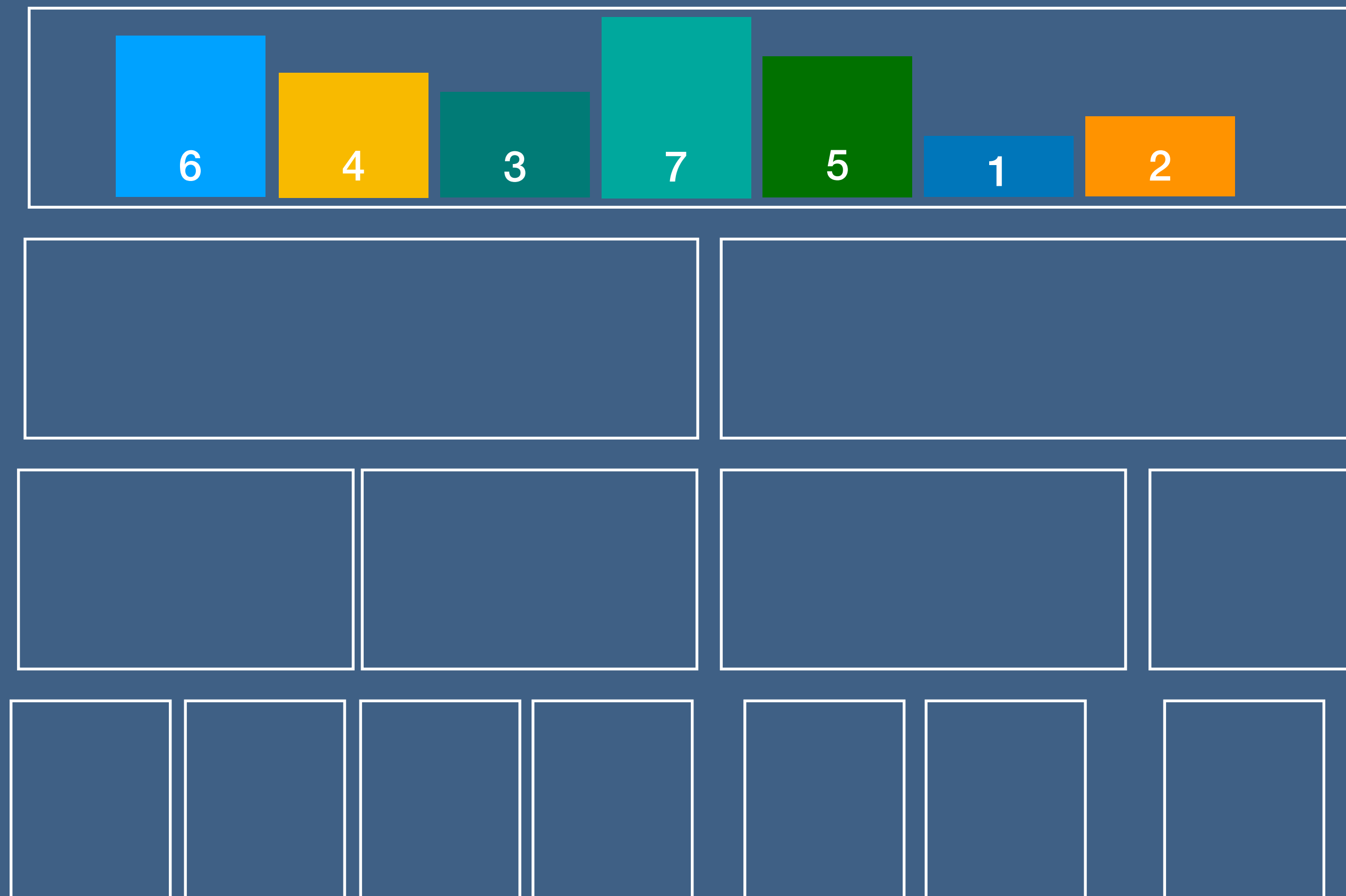


Merge Sort

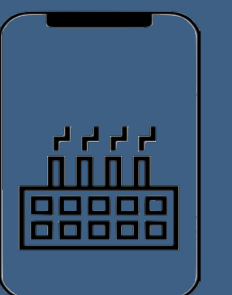
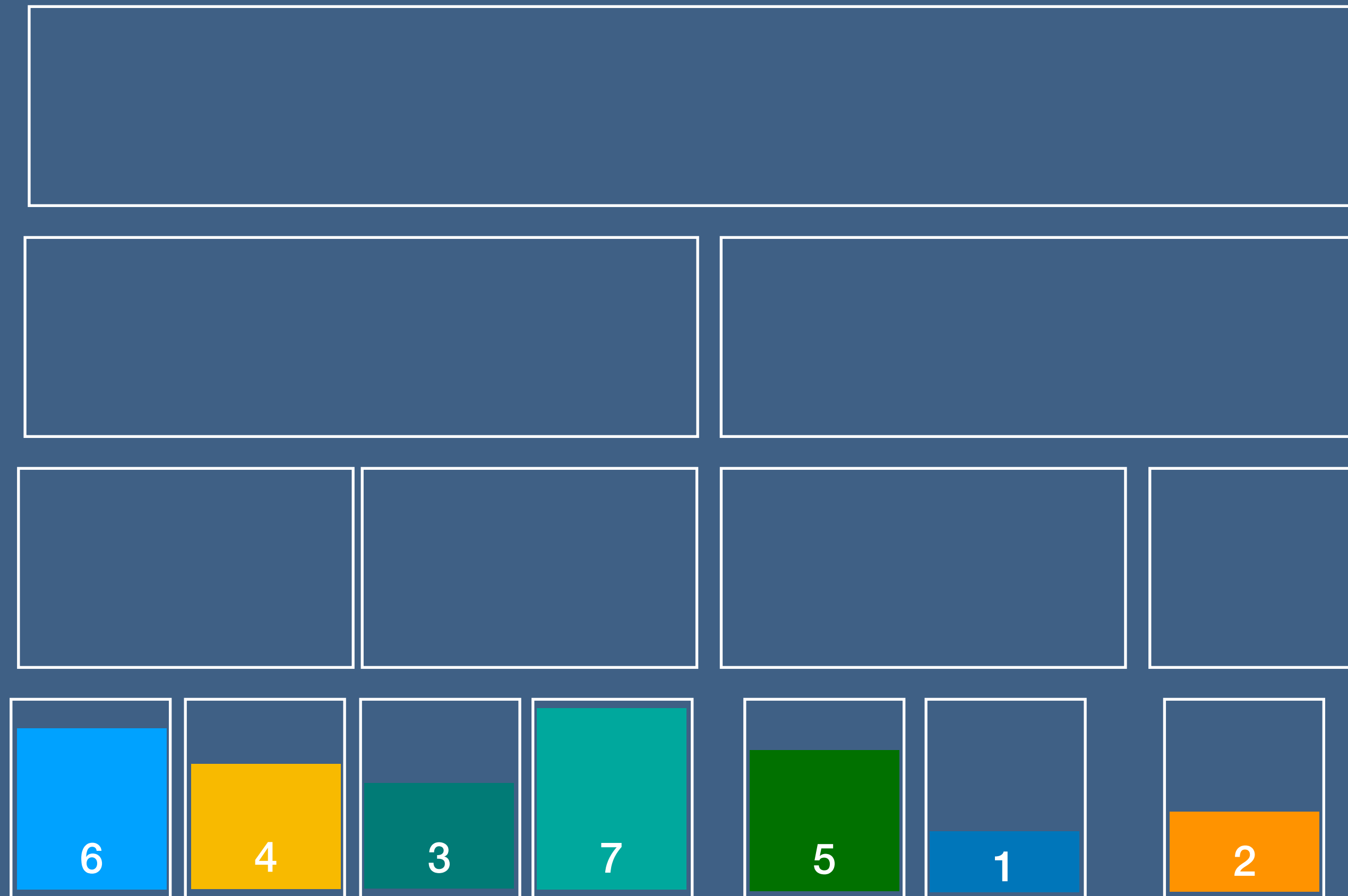
- Merge sort is a divide and conquer algorithm
- Divide the input array in two halves and we keep halving recursively until they become too small that cannot be broken further
- Merge halves by sorting them



Merge Sort



Merge Sort



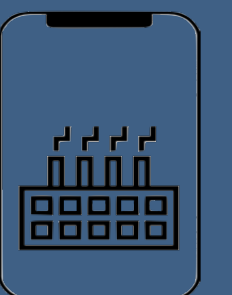
Merge Sort

When to use Merge Sort?

- When you need stable sort
- When average expected time is $O(N\log N)$

When to avoid Merge Sort?

- When space is a concern



Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required

70	10	80	30	90	40	60	20	50
----	----	----	----	----	----	----	----	----

10	30	40	20	50	80	60	70	90
----	----	----	----	----	----	----	----	----

10	20	40	30	50	80	60	70	90
----	----	----	----	----	----	----	----	----

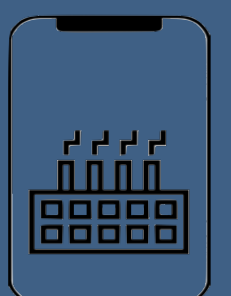
10	20	30	40	50	80	60	70	90
----	----	----	----	----	----	----	----	----

10	20	30	40	50	80	60	70	90
----	----	----	----	----	----	----	----	----

10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----

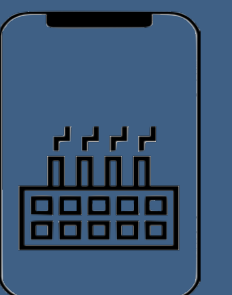
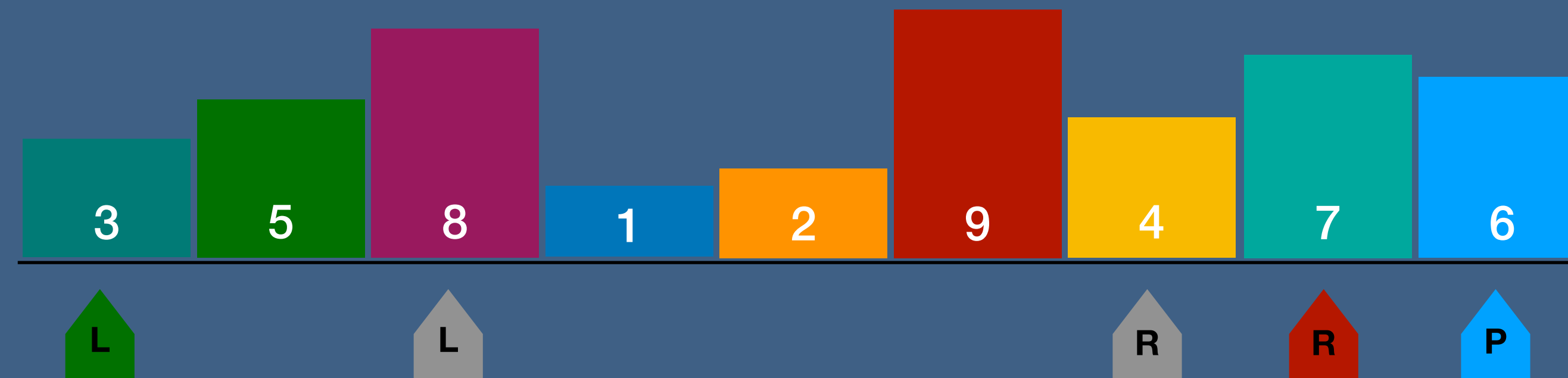
10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----

10	20	30	40	50	60	70	80	90
----	----	----	----	----	----	----	----	----



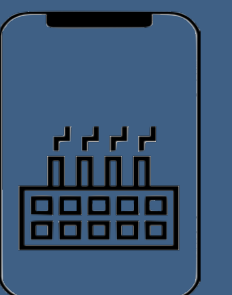
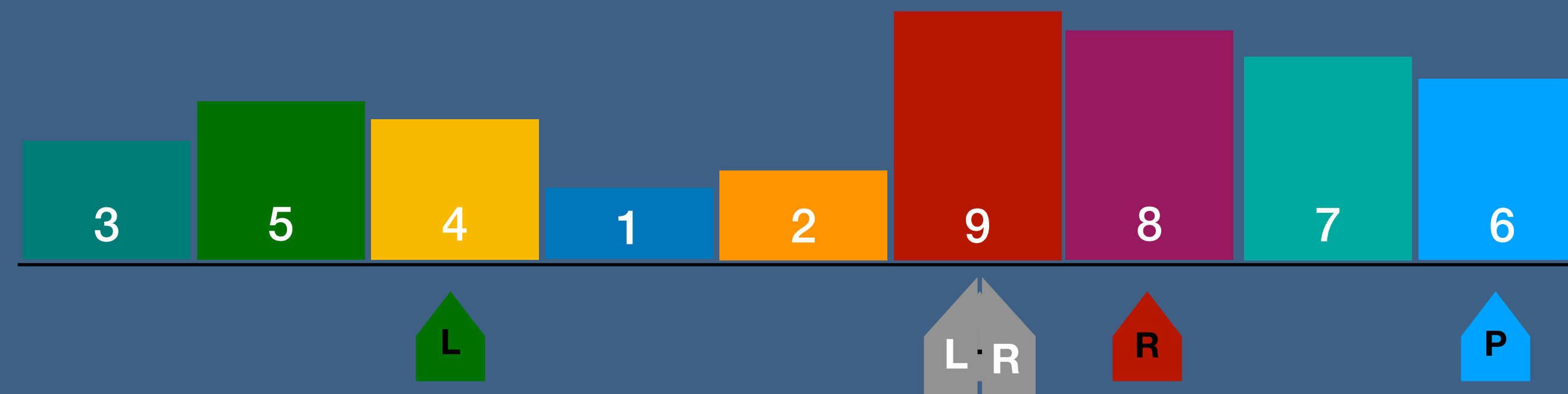
Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



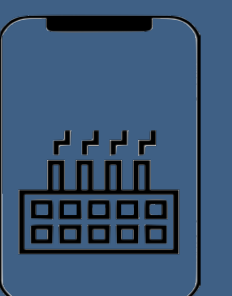
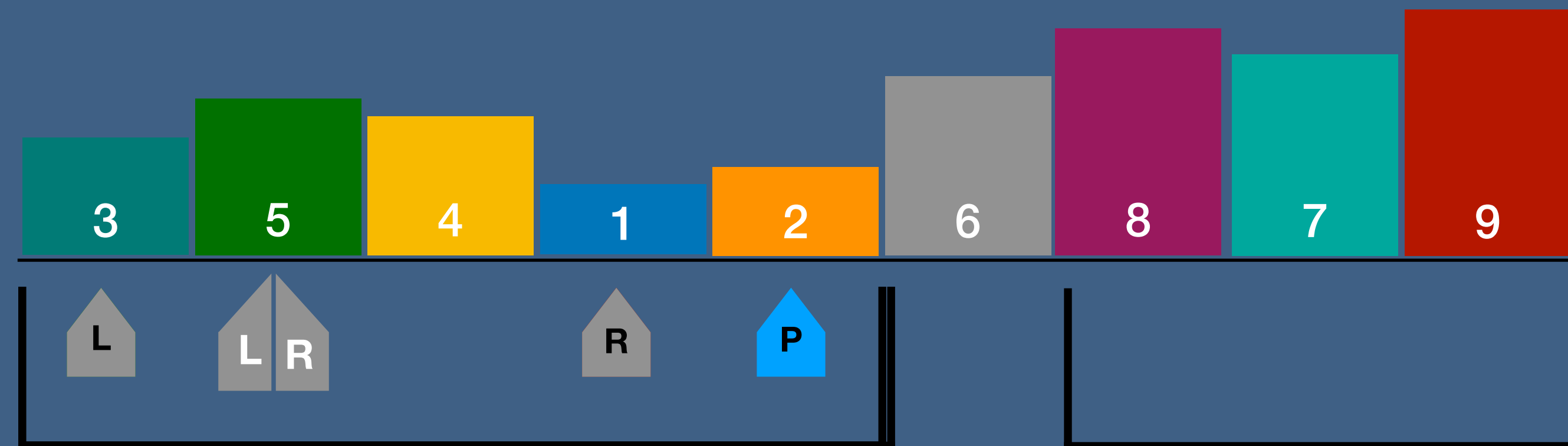
Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



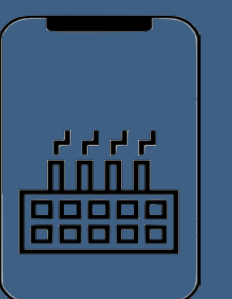
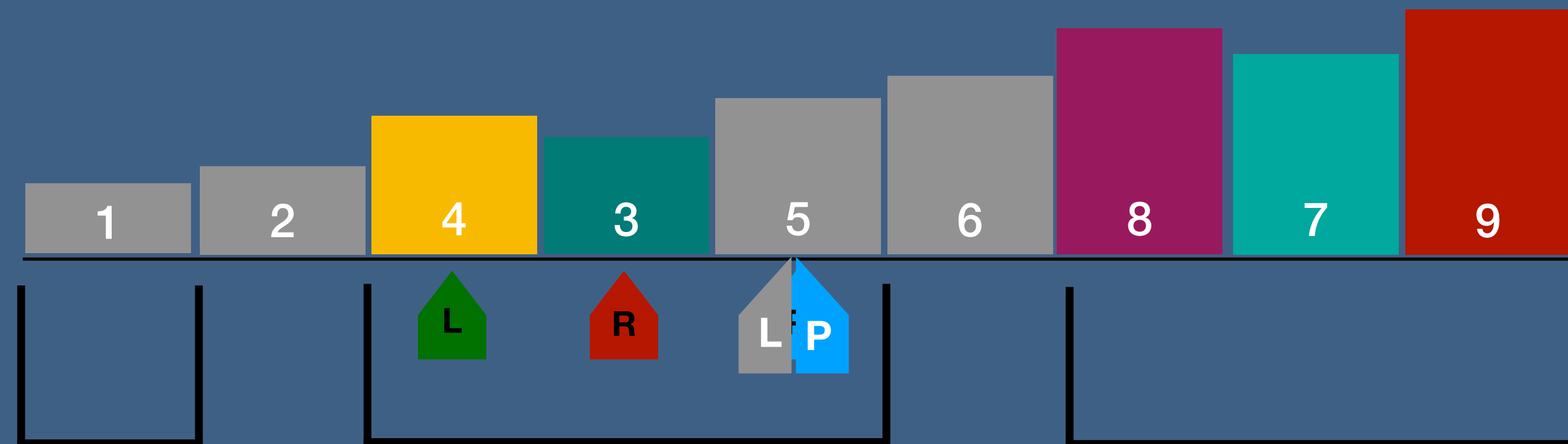
Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



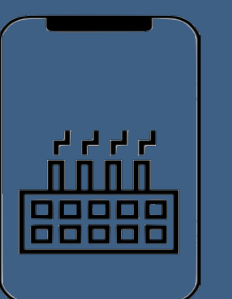
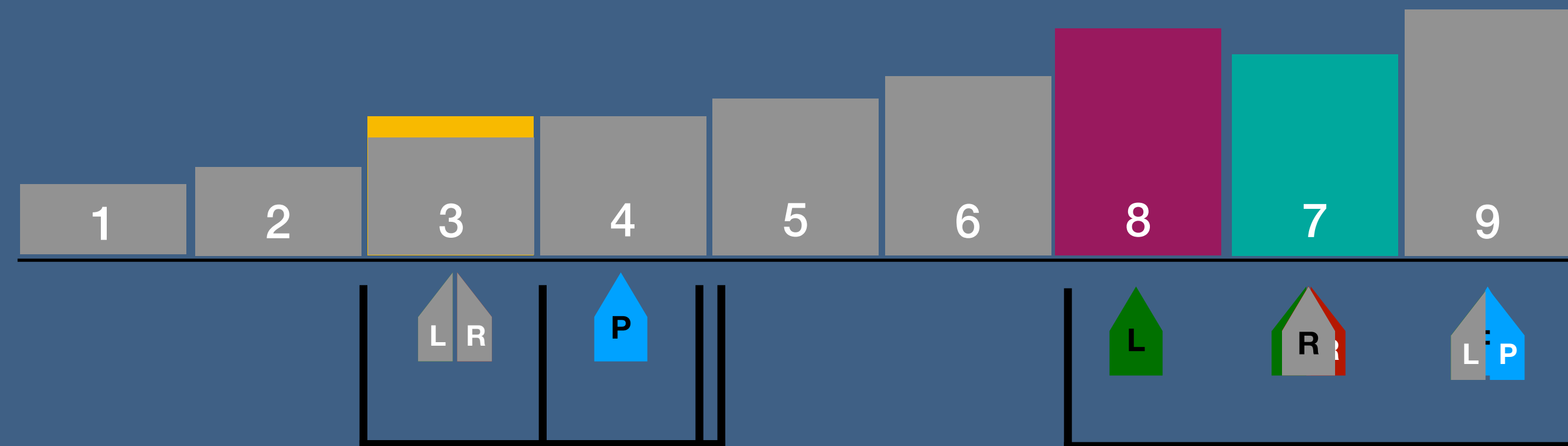
Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



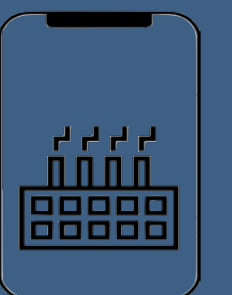
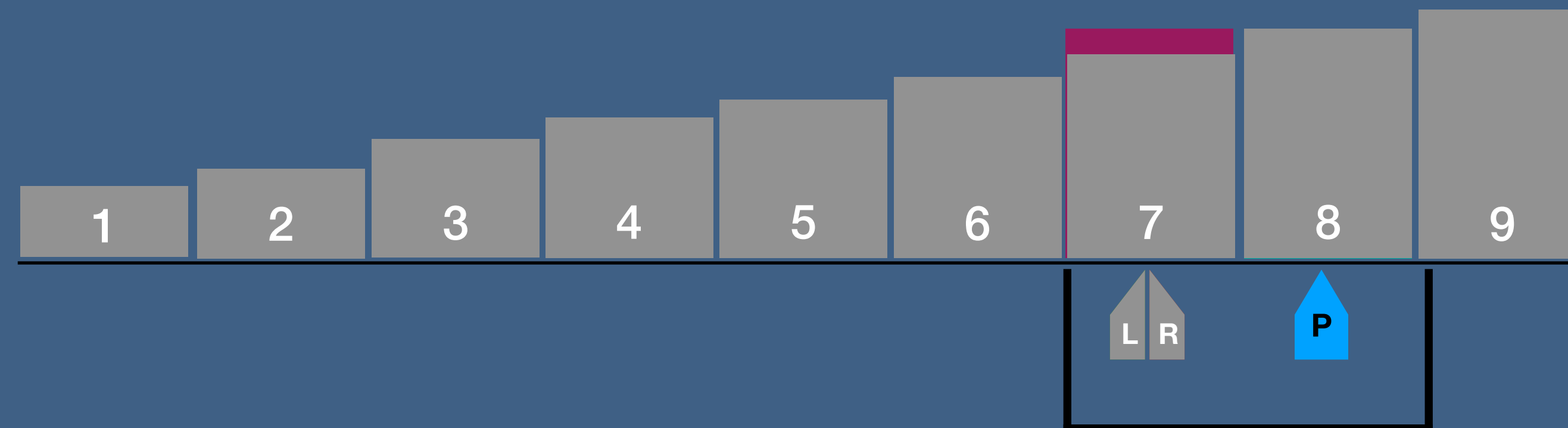
Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



Quick Sort

- Quick sort is a divide and conquer algorithm
- Find pivot number and make sure smaller numbers located at the left of pivot and bigger numbers are located at the right of the pivot.
- Unlike merge sort extra space is not required



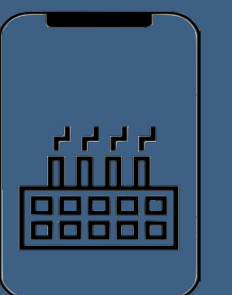
Quick Sort

When to use Quick Sort?

- When average expected time is $O(N\log N)$

When to avoid Quick Sort?

- When space is a concern
- When you need stable sort

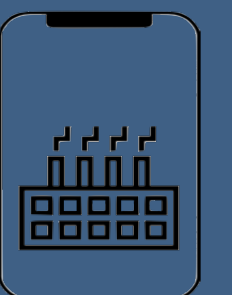
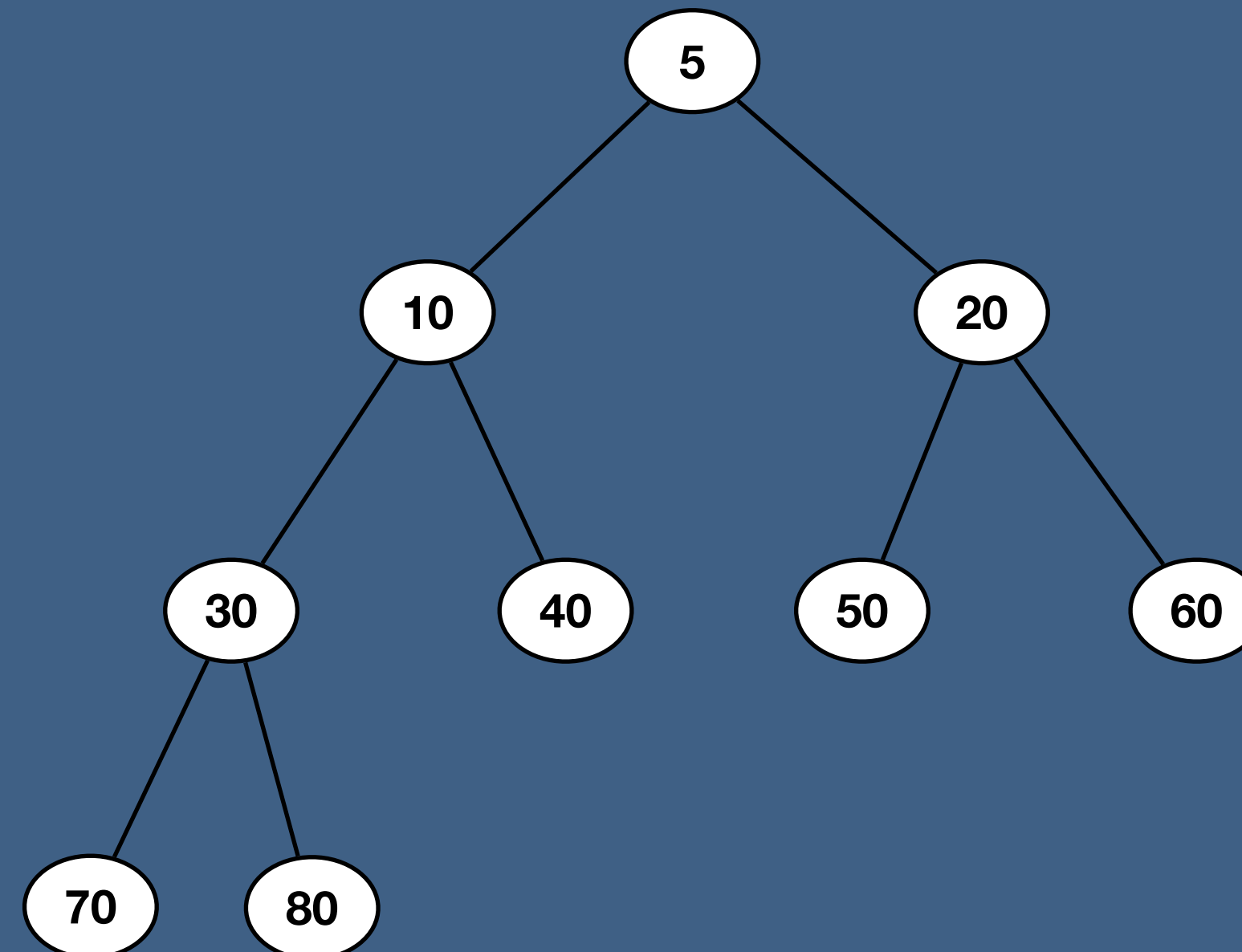


Heap Sort

- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap
- It is best suited with array, it does not work with Linked List

Binary Heap is a binary tree with special properties

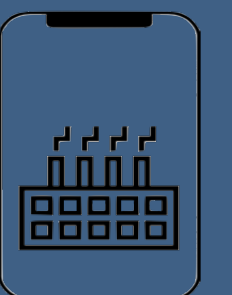
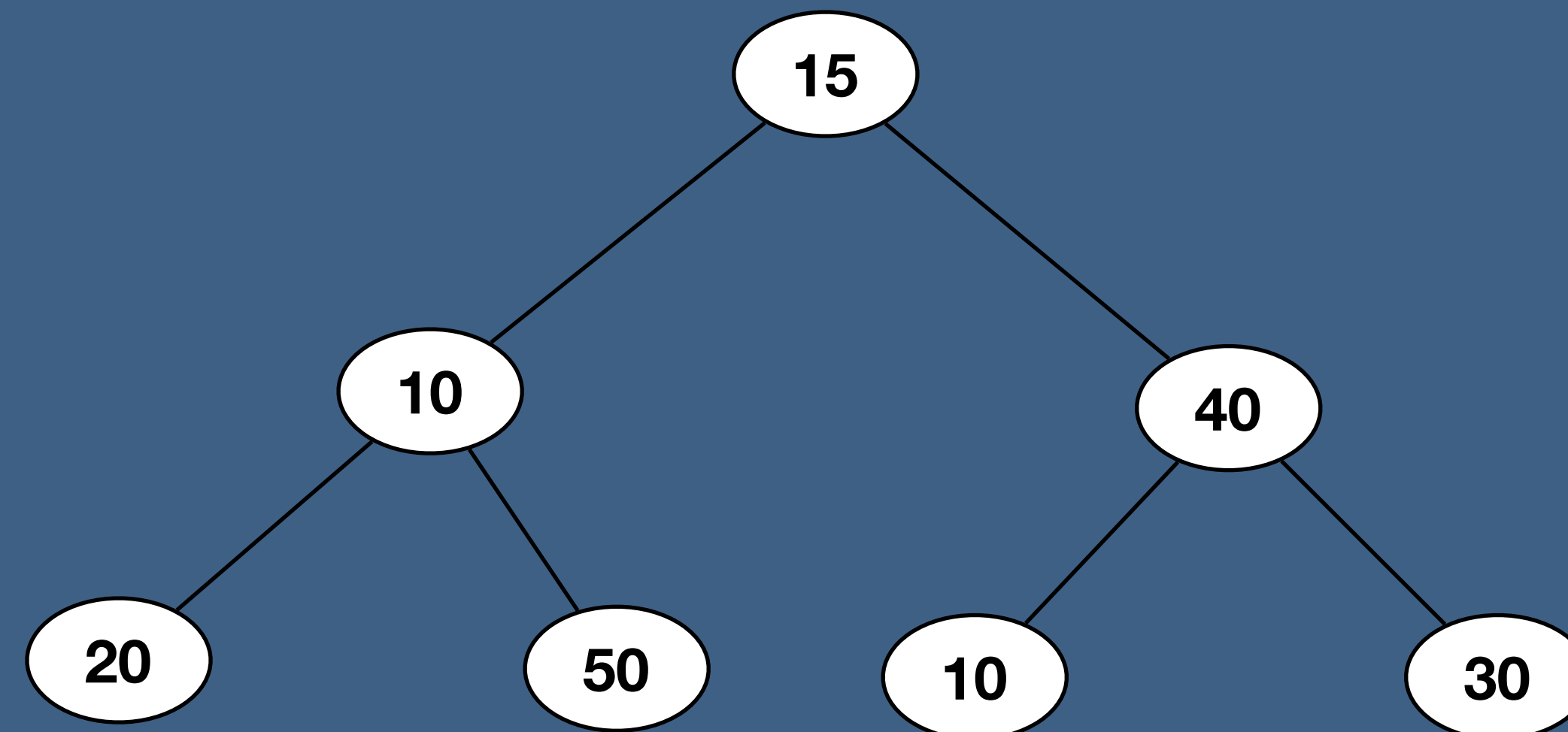
- The value of any given node must be less or equal of its children (min heap)
- The value of any given node must be greater or equal of its children (max heap)



Heap Sort

- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap

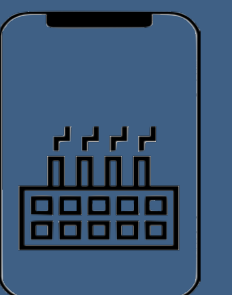
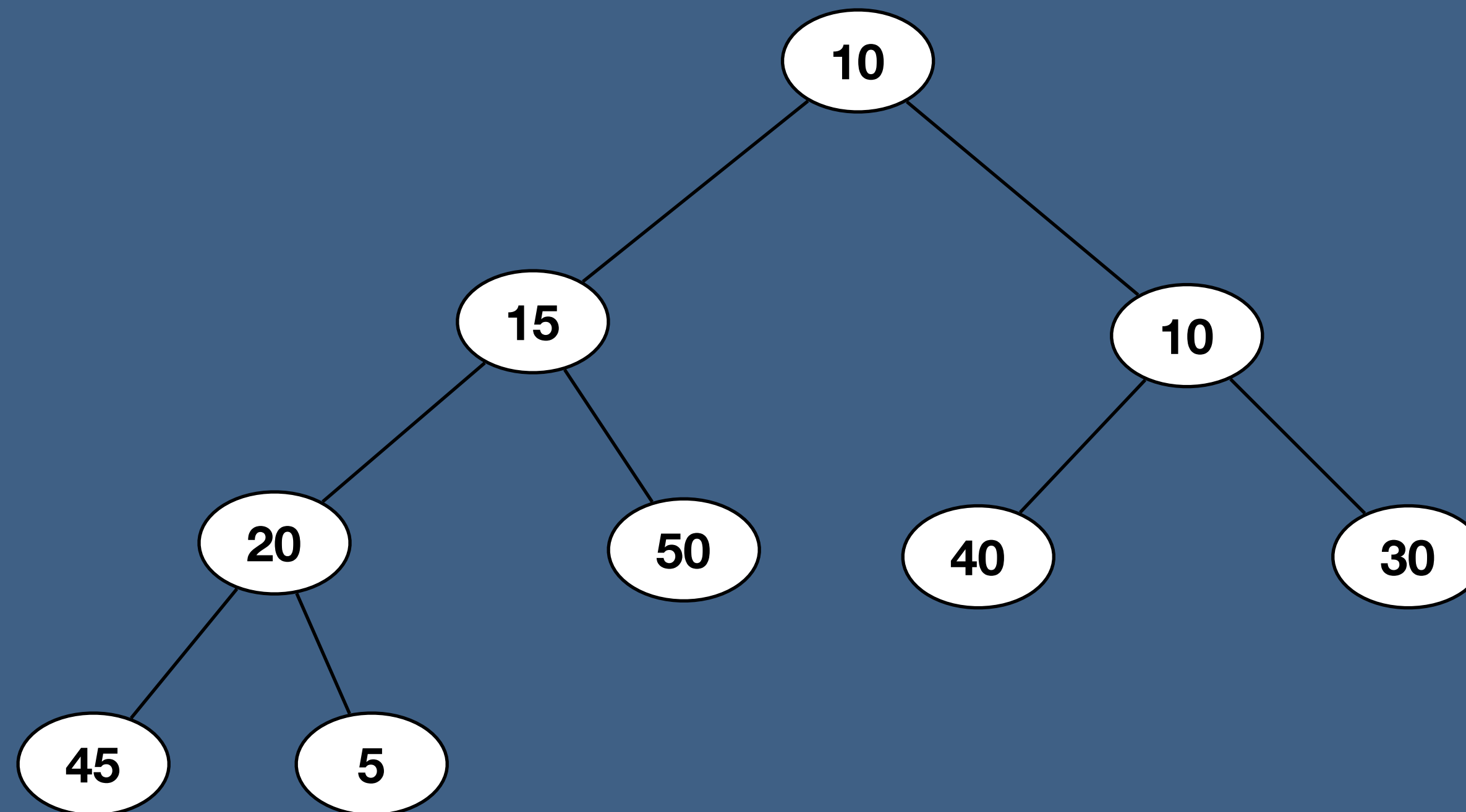
15	10	40	20	50	10	30	45	5
----	----	----	----	----	----	----	----	---



Heap Sort

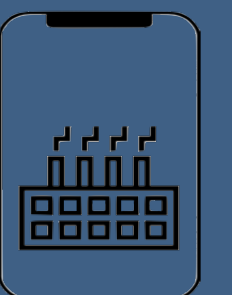
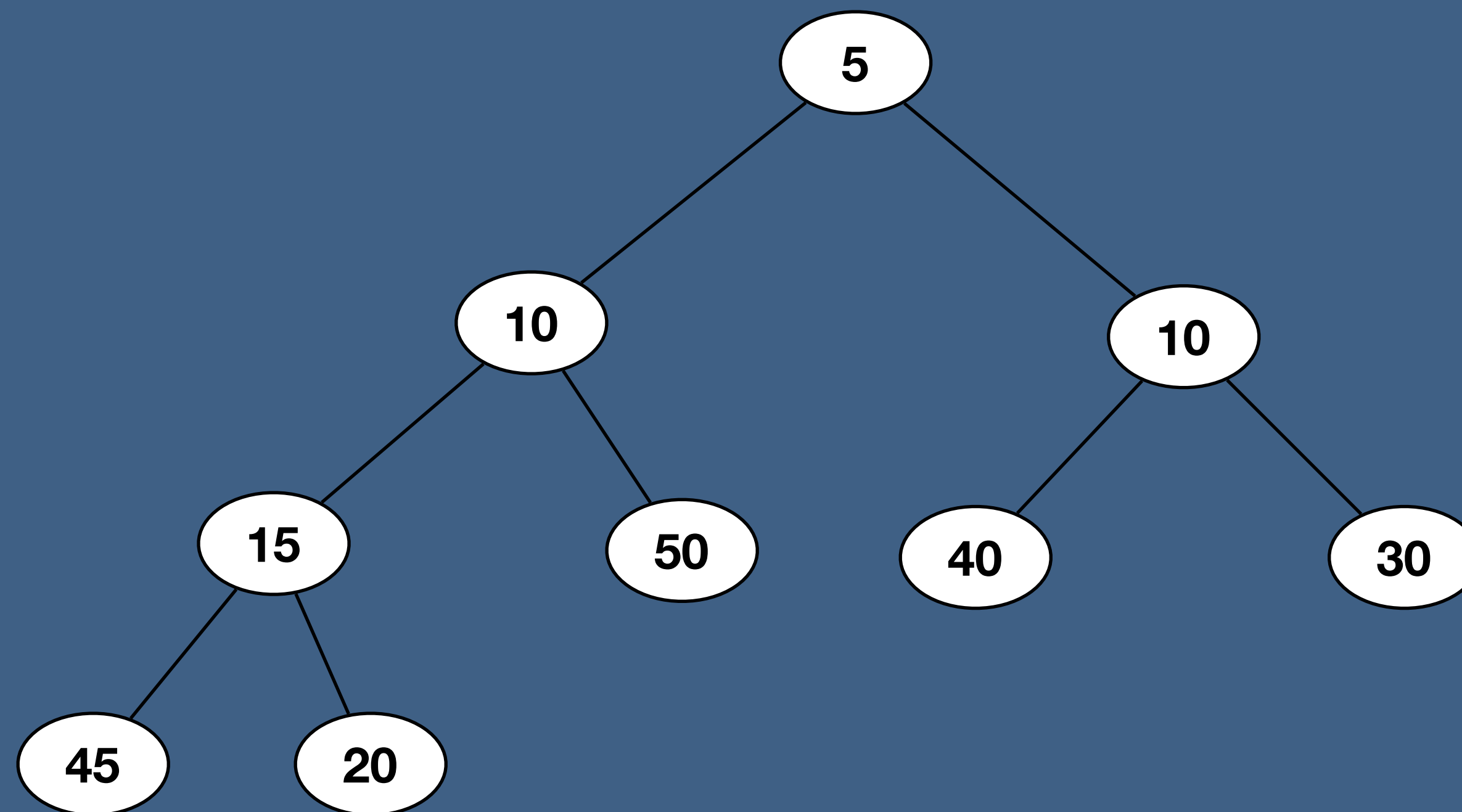
- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap

15	10	40	20	50	10	30	45	5
----	----	----	----	----	----	----	----	---



Heap Sort

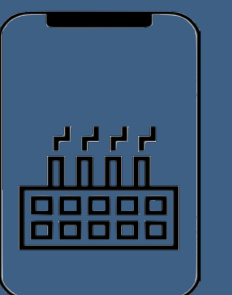
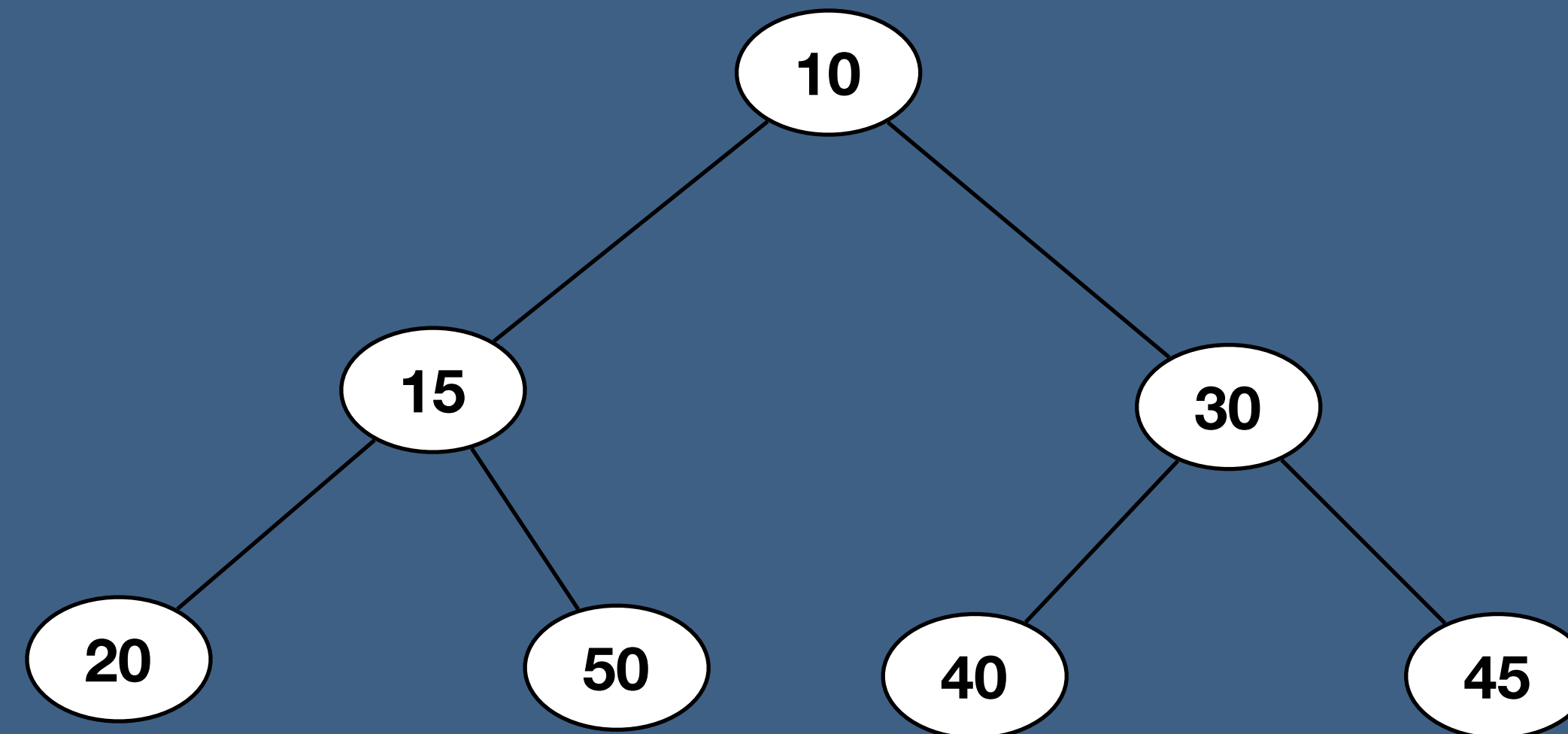
- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap



Heap Sort

- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap

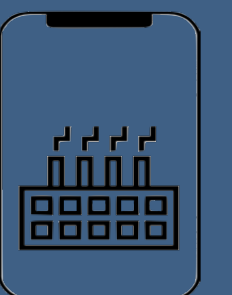
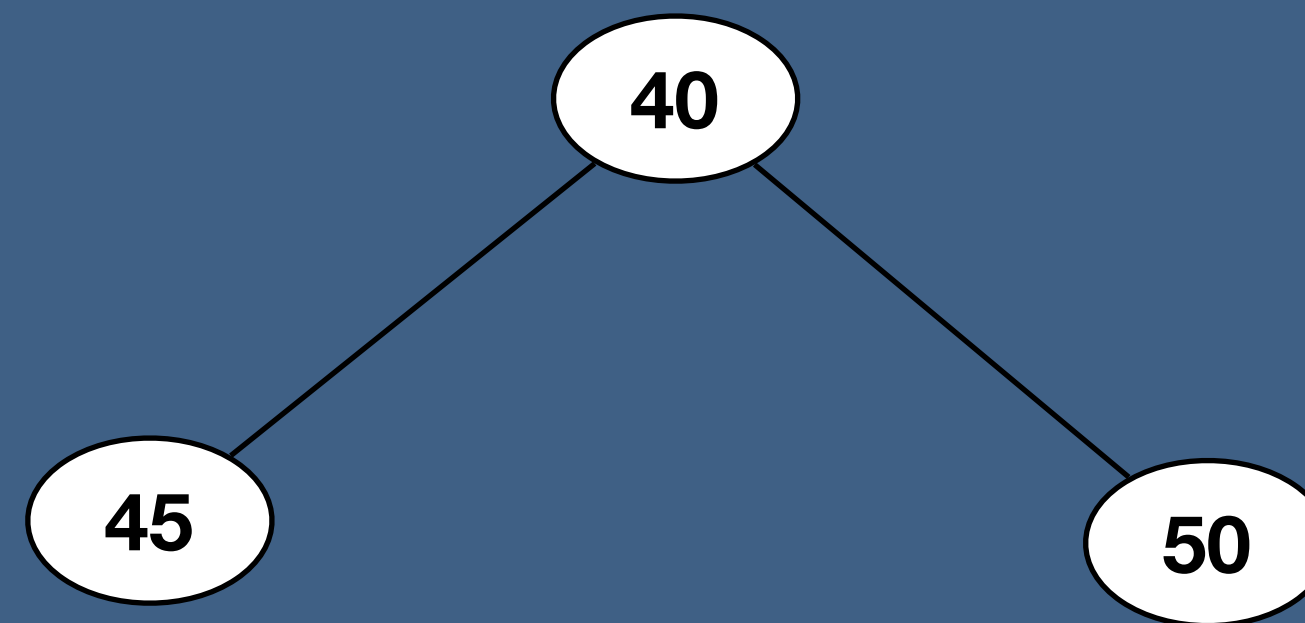
5	10	10	15	20	30			
---	----	----	----	----	----	--	--	--



Heap Sort

- Step 1 : Insert data to Binary Heap Tree
- Step 2 : Extract data from Binary Heap

5	10	10	15	20	30	40	45	50
---	----	----	----	----	----	----	----	----



Sorting Algorithms

Name	Time Complexity	Space Complexity	Stable
Bubble Sort	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n^2)$	$O(1)$	Yes
Bucket Sort	$O(n \log n)$	$O(n)$	Yes
Merge Sort	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n)$	No
Heap Sort	$O(n \log n)$	$O(1)$	No

